

EMBEDDED SYSTEM DESIGN

MODULE-6

**Emerging bus standards
and communications**

UART, SPI, I2C

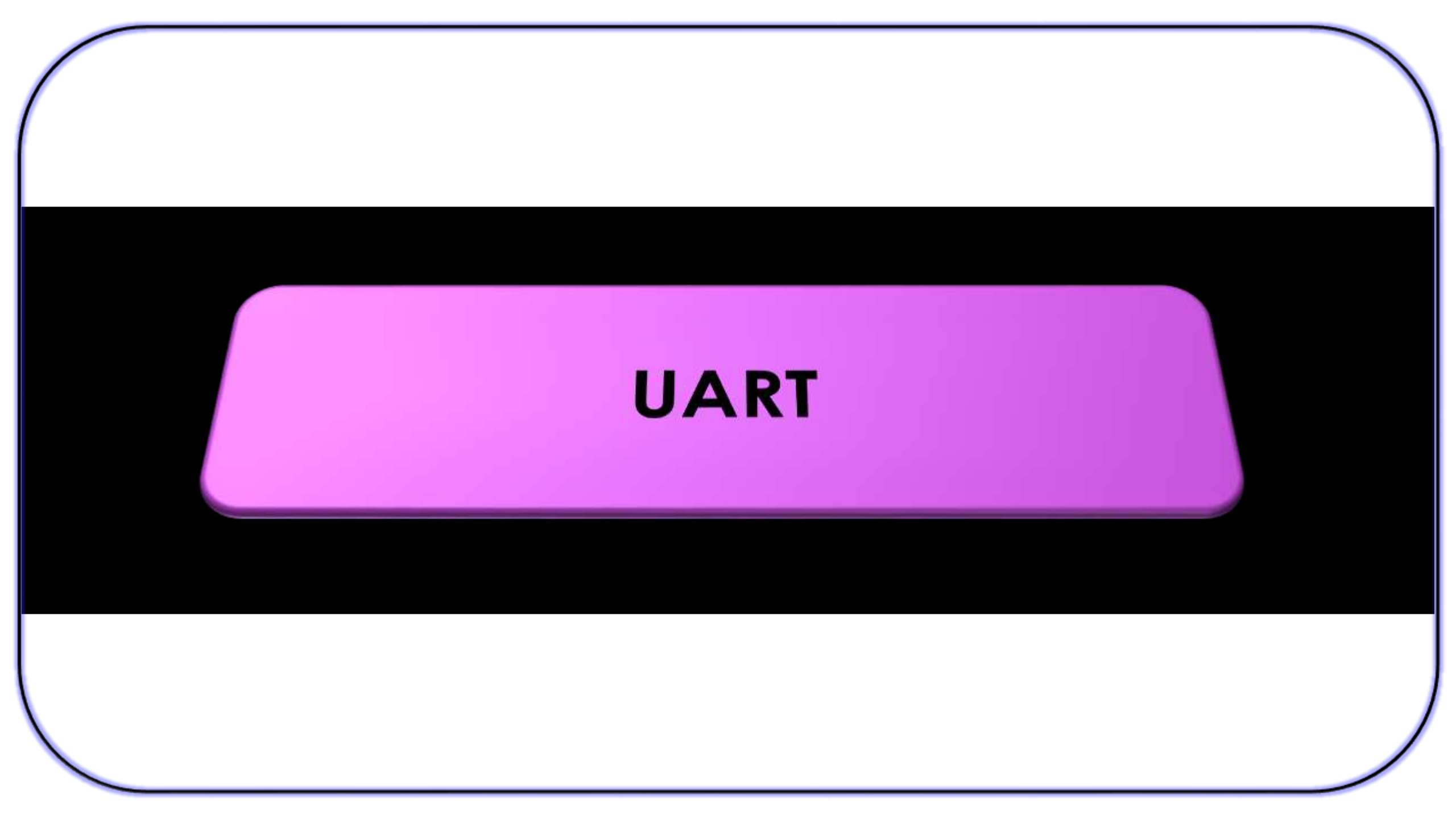
CAN, Ethernet

**BLUETOOTH, ZIGBEE,
WIFI**

COMMUNICATION PROTOCOLS

INTRODUCTION

- ❑ Why communication protocols and bus standards?
 - ❑ Formal format for message and rules for communication between devices
 - ❑ They cover authentication, error detection and correction, handling of signals
 - ❑ Standard bus are used for sending data, address and control signals between controllers and peripherals

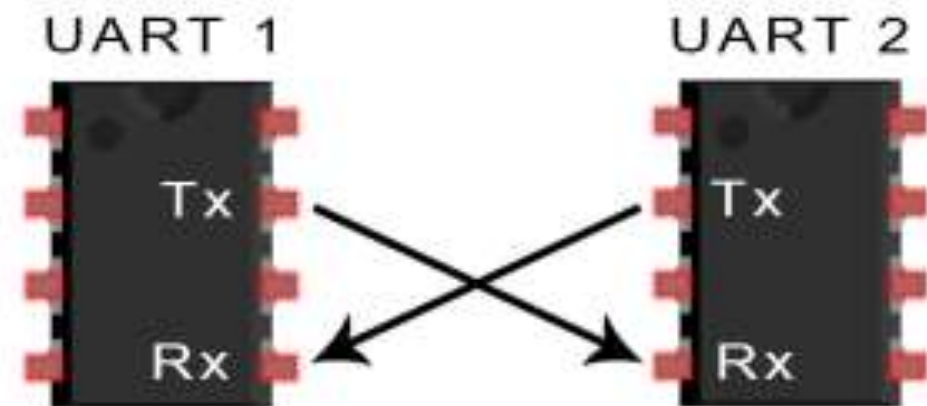


UART

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- ❑ **UART** stands for **Universal Asynchronous Receiver/Transmitter**
- ❑ It is used to transmit and receive serially
- ❑ Two wires are required to transmit and receive data **Tx** and **Rx**
Pin



UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- ❑ UARTs has no clock signal and follows asynchronous data communication
- ❑ Start and stop bits are added to data packet for identifying beginning and end of data packet
- ❑ Start bit and stop bit are used instead of clock signal

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

- ❑ After detecting start bit UART starts to read the incoming data packet
- ❑ A specific frequency is used for data communication known as baud rate
- ❑ Baud rate is expressed as bits per second (bps). It is a measure of speed of data transfer.
- ❑ Both receiving and transmitting UART should work in same baud rate

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

INTRODUCTION

Parameters	Specification
Wires used	2
Maximum speed	115200 baud rate
Synchronous ?	No
Serial?	Yes
Max number of master	1
Max number of slave	1

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS

- ❑ In UART, data is transmitted in parallel from data bus to transmitting UART
- ❑ The transmitting UART adds a start bit, stop bit and a parity bit to parallel data from data bus
- ❑ the data packet is transmitted serially from Tx to RX pin
- ❑ The Rx pin reads the data and converts parallelly by removing other bits

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS

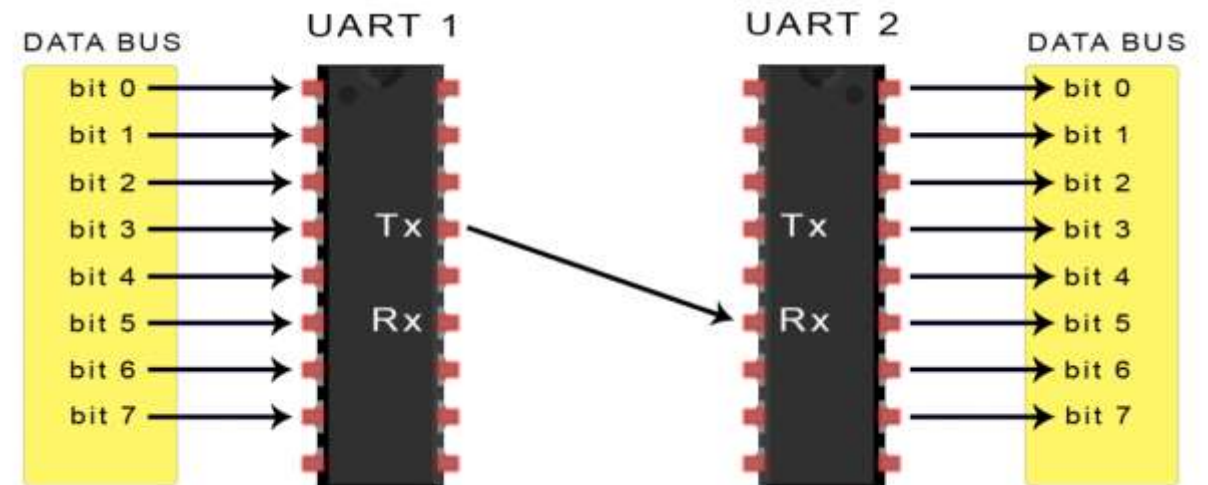
Frame Format

1 start bit

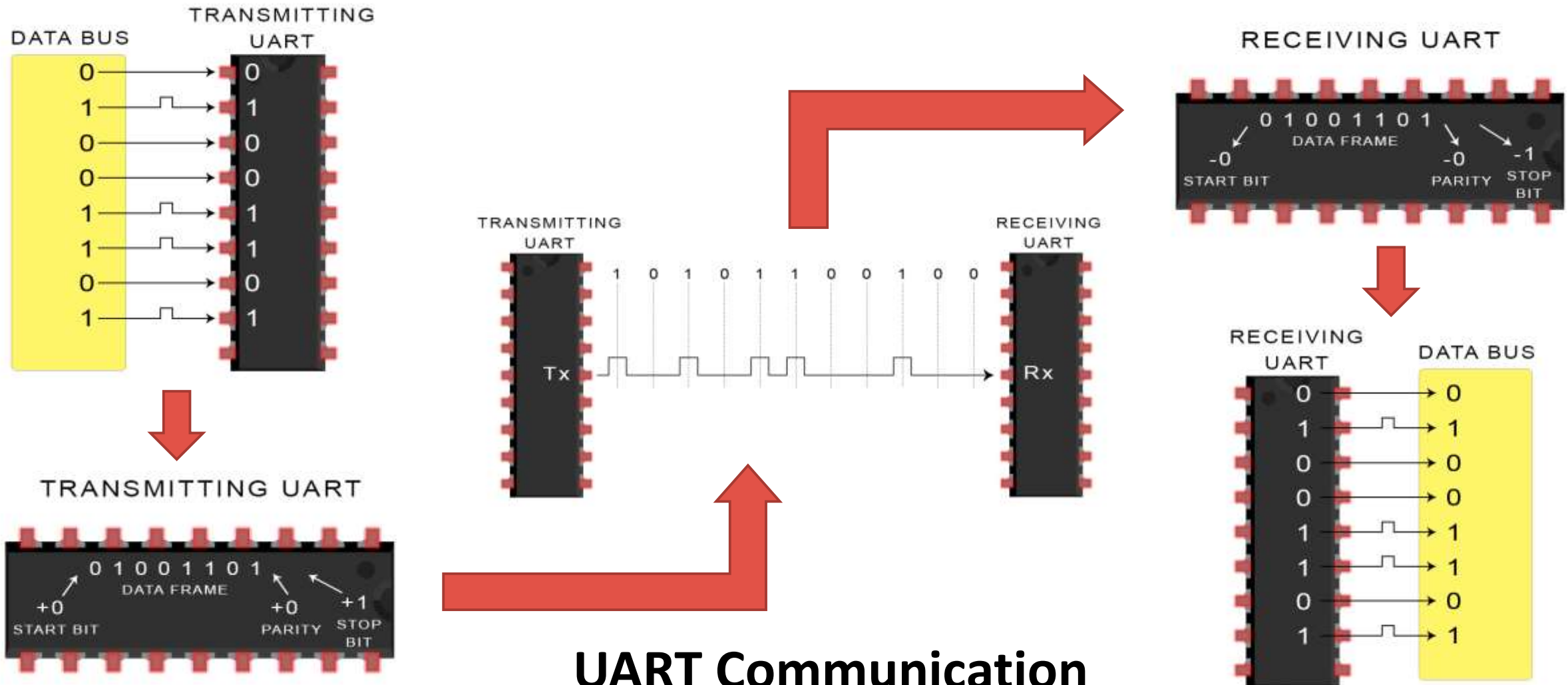
5 to 9 data
bit

0 or 1 parity
bit

1 to 2 stop
bits



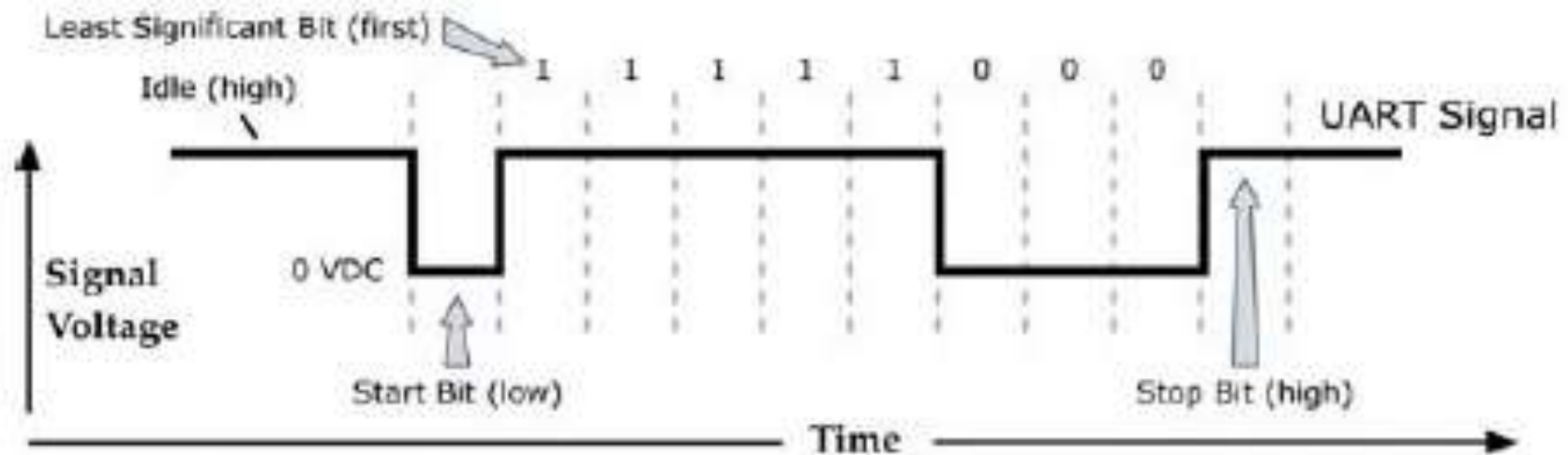
UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER



UART Communication

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

HOW UART WORKS



A common asynchronous serial data format

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

UART IN ARDUINO

- ❑ UART allows the Atmega chip to do serial communication while working on other tasks, through 64 byte serial buffer.
- ❑ All Arduino boards have at least one serial port (also known as a UART or USART): **Serial**.
- ❑ Serial is used for communication between the Arduino board and a computer or other devices.
- ❑ It communicates on digital pins 0 (RX) and 1 (TX) as well as with the computer via USB. Thus, if you use these functions, you cannot also use pins 0 and 1 for digital input or output.

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino functions for USB serial communication

- `if (Serial)` – indicates whether or not the USB serial connection is open
- `Serial.available()` -Get the number of bytes available for reading from the serial port.
- `Serial.begin()` -Sets the data rate in bits per second (baud) for serial data transmission.
- `Serial.end()` -Disables serial communication, allowing the RX,TX to be used for input & output.
- `Serial.find()` -reads data from the serial buffer until the target is found.
- `Serial.println()` -Prints data as ASCII text followed by a carriage return and newline character.
- `Serial.read()` -Reads incoming serial data.
- `Serial.readString()` - reads characters from the serial buffer into a String.
- `Serial.write()` - Writes binary data to serial port. This data is sent as a byte or series of bytes

For other functions refer the URL:

<https://www.arduino.cc/en/pmwiki.php?n=Reference/serial#:~:text=Serial%20is%20used%20for%20communication,with%20the%20computer%20via%20USB.>

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- ❑ **EXAMPLE-1:** Print data received through serial communication on to the serial monitor of Arduino

```
void setup() {  
  Serial.begin(9600); //set up serial library baud rate to 9600  
}  
  
void loop() {  
  if(Serial.available()) //if number of bytes (characters) available for reading from serial port  
  {  
    Serial.print("I received:"); //print I received  
    Serial.write(Serial.read()); //send what you read  
  }  
}
```

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- ❑ **EXAMPLE-1: Arduino code for serial interface to blink switch ON LED when “a” is received on serial port**

```
int inByte; // Stores incoming command
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT); // Led pin
  Serial.println("Ready"); // Ready to receive commands
}
void loop() {
  if(Serial.available() > 0) { // A byte is ready to receive
    inByte = Serial.read();
    if(inByte == 'a') { // byte is 'a'
      digitalWrite(13, HIGH);
      Serial.println("LED - On");
    }
    else { // byte isn't 'a'
      digitalWrite(13, LOW);
      Serial.println("LED - off");
    }
  }
}
```

This function can be used
if(Serial.find("a"))

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino Software Serial Library

- ❑ The SoftwareSerial library has been developed to allow serial communication on other digital pins of the Arduino
- ❑ Uses software to replicate the functionality of the hardwired RX and TX lines hence the name "SoftwareSerial".
- ❑ It is possible to have multiple software serial ports with speeds up to 115200 bps.
- ❑ This can be extremely helpful when the need arises to communicate with two or more serial enabled devices
- ❑ Limitations:
 - maximum RX speed is 57600bps and RX doesn't work on Pin 13
 - If using multiple software serial ports, only one receive data at a time.

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

Arduino functions for UART communication (SoftwareSerial Library)

Software Serial() – Need to enable serial communication

available() – gets the no of bits available for reading from serial port

begin() – sets the speed of serial communication

overflow() – checks the serial buffer overflow has occurred

peek() – returns the character received in the serial port

read() - returns the character that was received on the Rx pin of serial port

Print() – works same as serial.print

Listen() – enables the selected serial port to listen

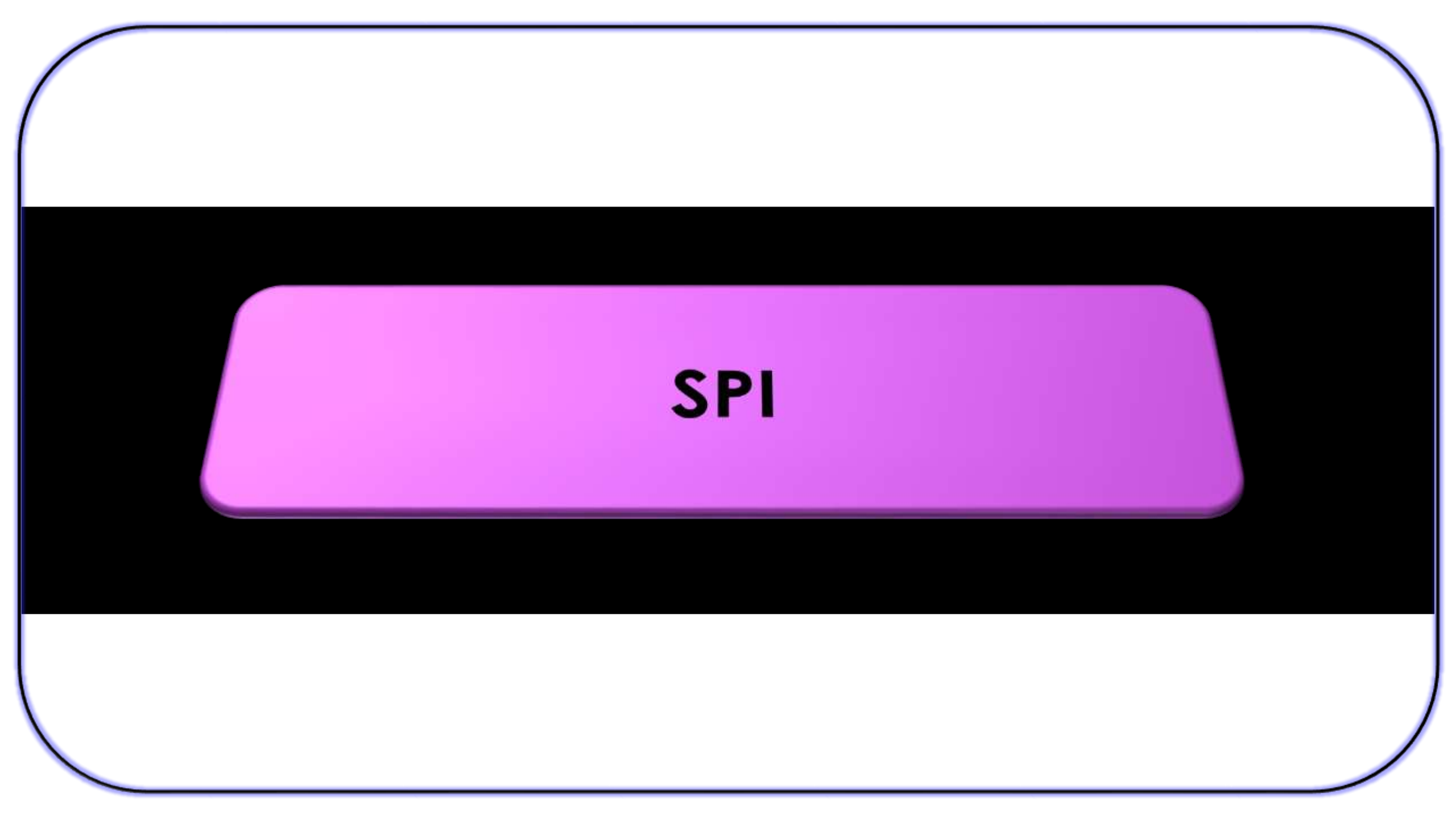
Write() – print data to transmit pin of software serial

Refer. URL: <https://www.arduino.cc/en/Reference/SoftwareSerial>

UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER

- ❑ **EXAMPLE-3: Arduino code to Receives from the hardware serial, sends to software serial and Receives from software serial, sends to hardware serial.**

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX
void setup() {
    Serial.begin(57600); // Open serial communications and wait for port to open:
    while (!Serial) {
        ; // wait for serial port to connect. Needed for native USB port only
    }
    mySerial.begin(4800); // set the data rate for the SoftwareSerial port
    mySerial.println("Hello, world?");
}
void loop() { // run over and over
    if (mySerial.available()) {
        Serial.write(mySerial.read());
    }
    if (Serial.available()) {
        mySerial.write(Serial.read());
    }
}
```



SPI

SERIAL PERIPHERAL INTERFACE

INTRODUCTION

- ❑ Serial Peripheral Interface or SPI is a full-duplex serial communication protocol
- ❑ It works in synchronous mode at very high speed
- ❑ SPI is more appropriate for short distance communication with a particular device or on-board devices
- ❑ It is available in most of the microcontrollers like PIC, AVR, ARM and so on.

SERIAL PERIPHERAL INTERFACE

INTRODUCTION

- ❑ Master-slave model of communication is supported by SPI
- ❑ SPI facilitates low cost and flexible interface between processor and peripherals
- ❑ For example, Microcontroller act as master and other devices like GSM, GPS, Sensors act as slave
- ❑ But it does not support multi-master communication

SERIAL PERIPHERAL INTERFACE

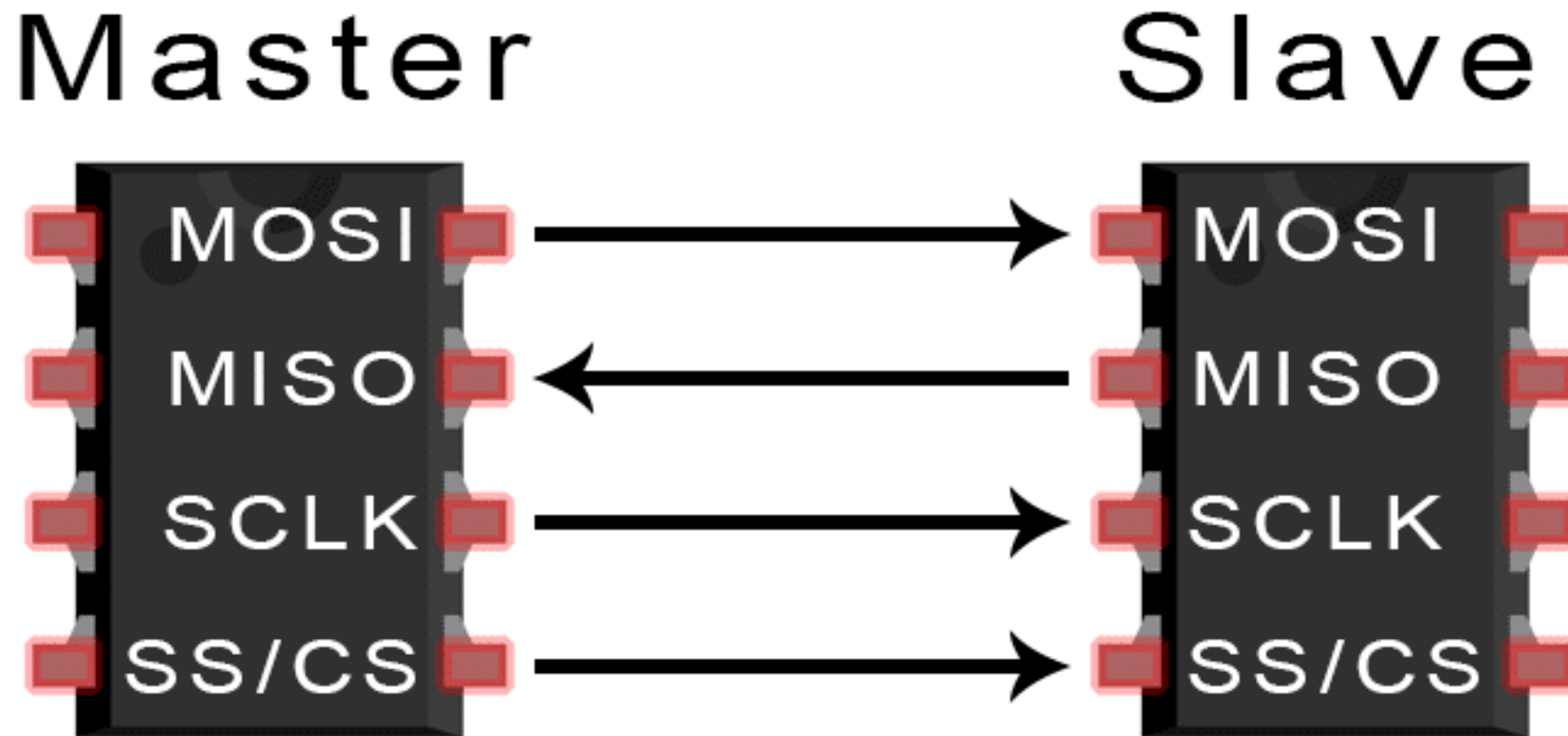
INTRODUCTION

The SPI bus consists of 4 signals or pins. They are

- Master – Out / Slave – In (MOSI): Master generates data and slave receives
- Master – In / Slave – Out (MISO): Slave generate data and transmit to Master.
- Serial Clock (SCLK): The Master generates clock signal and supplies to the clock input of slave.
- Chip Select (CS) or Slave Select (SS): Master selects a particular slave using chip select.

SERIAL PERIPHERAL INTERFACE

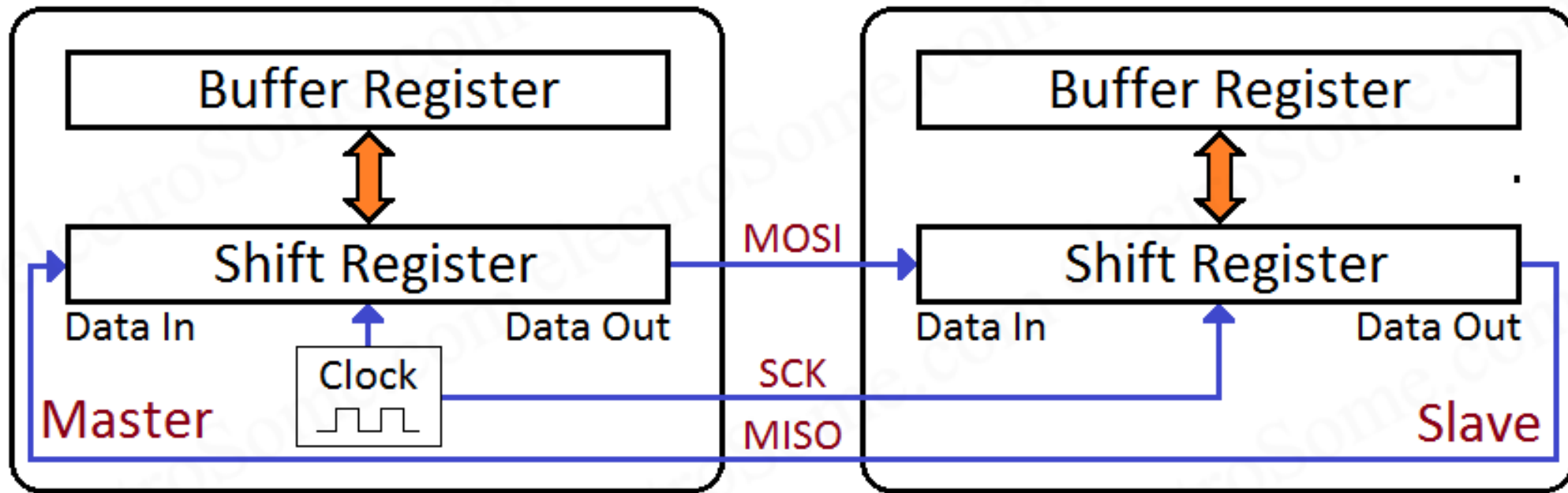
INTRODUCTION



Source: <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>

SERIAL PERIPHERAL INTERFACE

INTRODUCTION



SERIAL PERIPHERAL INTERFACE

INTRODUCTION

Parameters	Specification
Wires used	4
Maximum speed	10 Mbps
Synchronous ?	Yes
Serial?	Yes
Max number of master	1
Max number of slave	Theoretically unlimited

SERIAL PERIPHERAL INTERFACE

HOW SPI WORKS

❑ CLOCK:

- The clock synchronizes the data bits from master to the slave
- Only one bit is transmitted in each clock signal
- Frequency of the clock signal determines the data transfer rate
- Master initiates the communication in SPI and generates the clock signal

SERIAL PERIPHERAL INTERFACE

HOW SPI WORKS

❑ SLAVE SELECT (SS):

- Ideally slave select pin is kept at high voltage level
- The master decides which slave it wants to talk, by setting that slave's SS as low voltage level (one slave SS at a time)
- Master has many SS/CS pins, so that many slaves can be connected in parallel
- If only single SS pin available, then multiple slave can be connected to master by daisy chain configuration

SERIAL PERIPHERAL INTERFACE

HOW SPI WORKS

❑ MOSI AND MISO:

- The master sends data serially through MOSI line and slaves receives at MOSI pin
- Slave communicates serially back to master using MISO pin
- In master to slave communication MSB bit is first sent
- In slave to master communication LSB bit is first sent

SERIAL PERIPHERAL INTERFACE

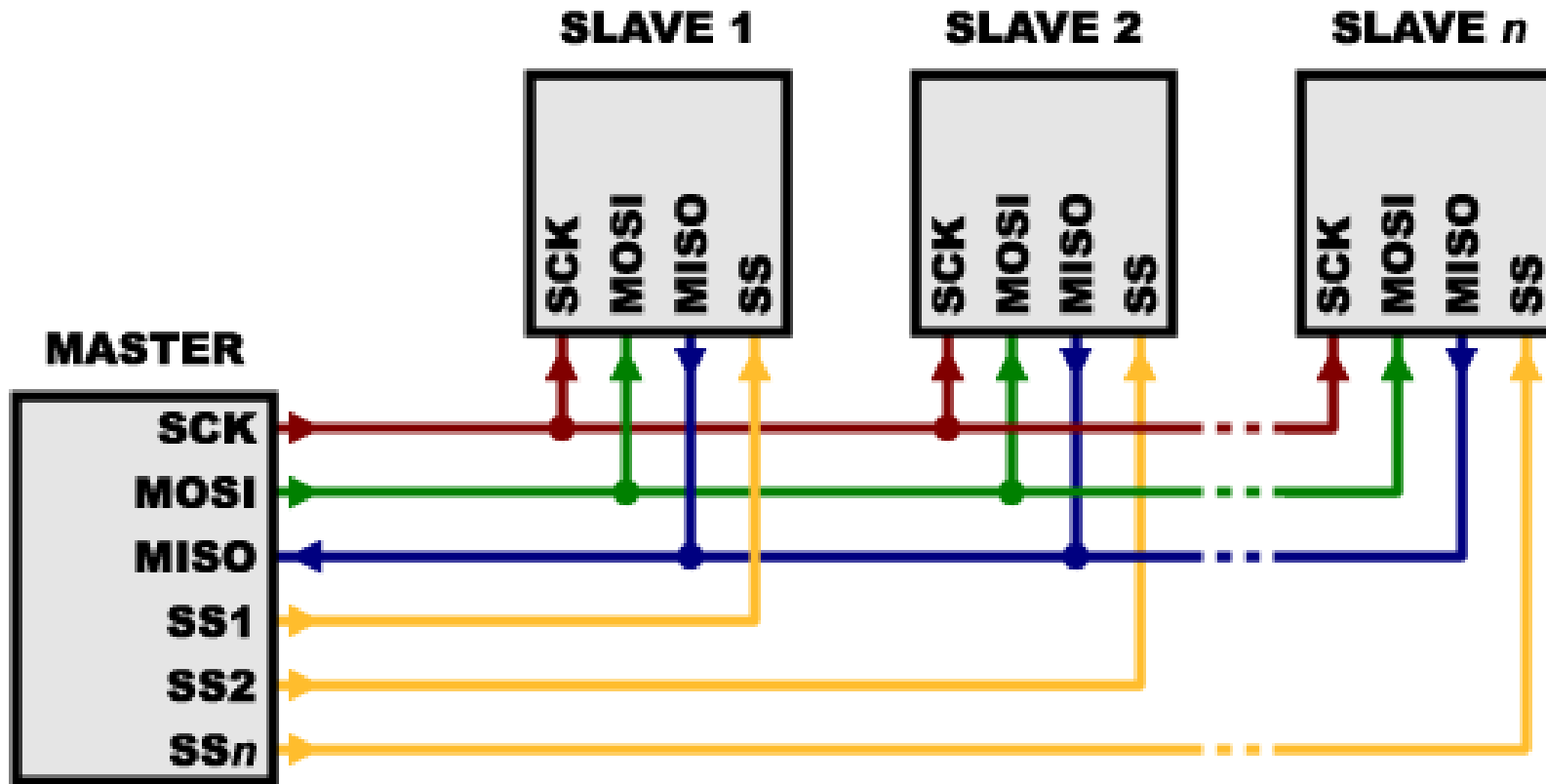
SPI CONFIGURATIONS

❑ Independent Slave Configuration:

- In independent configuration, all slaves have dedicated select lines and are connected individually to master
- Master SCK is connected to all the slave clock lines
- MOSI and MISO pins of both master and slaves are connected to each other

SERIAL PERIPHERAL INTERFACE

SPI CONFIGURATIONS



Source: <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>

Independent Slave Configuration

SERIAL PERIPHERAL INTERFACE

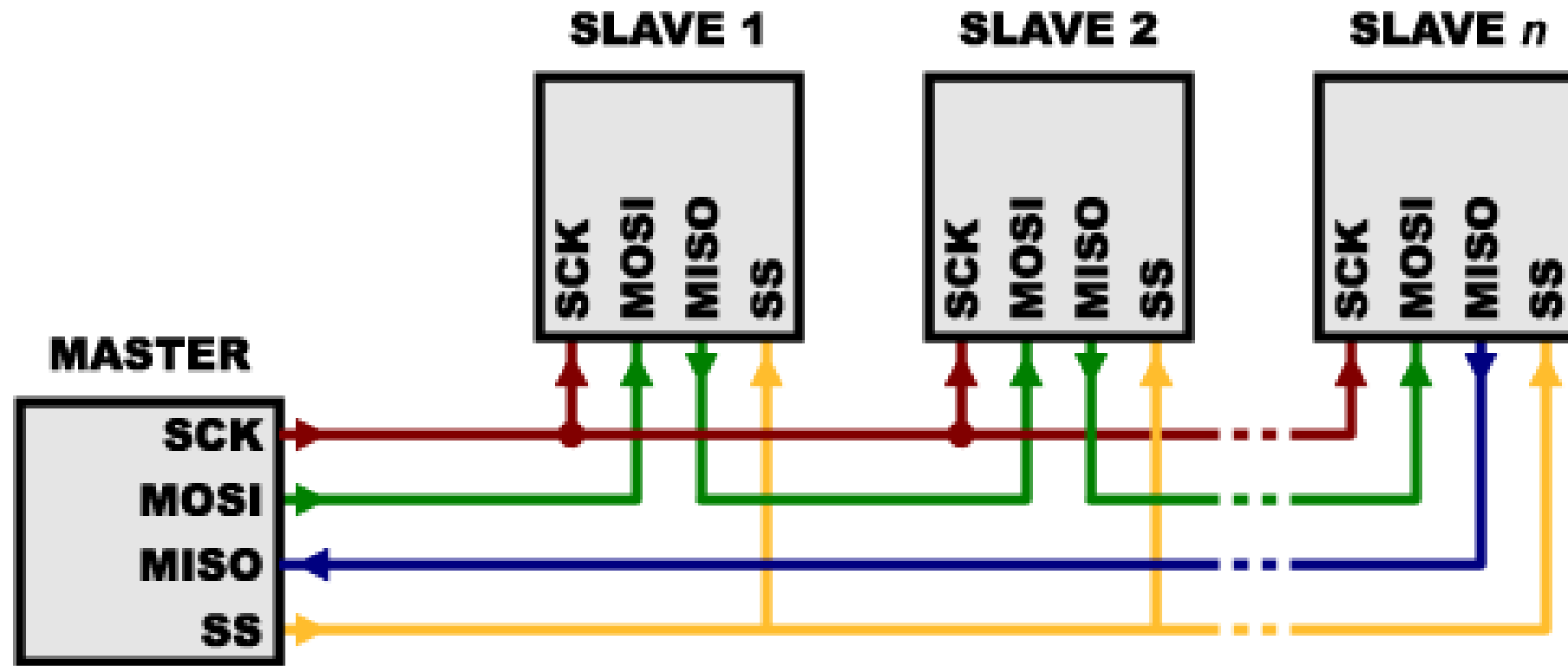
SPI CONFIGURATIONS

❑ Daisy Chain Configuration:

- In Daisy Chain Configuration, only a single Slave Select line is connected to all the slaves
- Only single slave select (SS) line is connected to all slaves
- The master MOSI pin is connected to Slave1 MOSI pin, Slave1 MISO pin is connected to Slave2 MOSI pin and so on.
- Finally, MISO of final slave is connected to MISO of master.

SERIAL PERIPHERAL INTERFACE

SPI CONFIGURATIONS



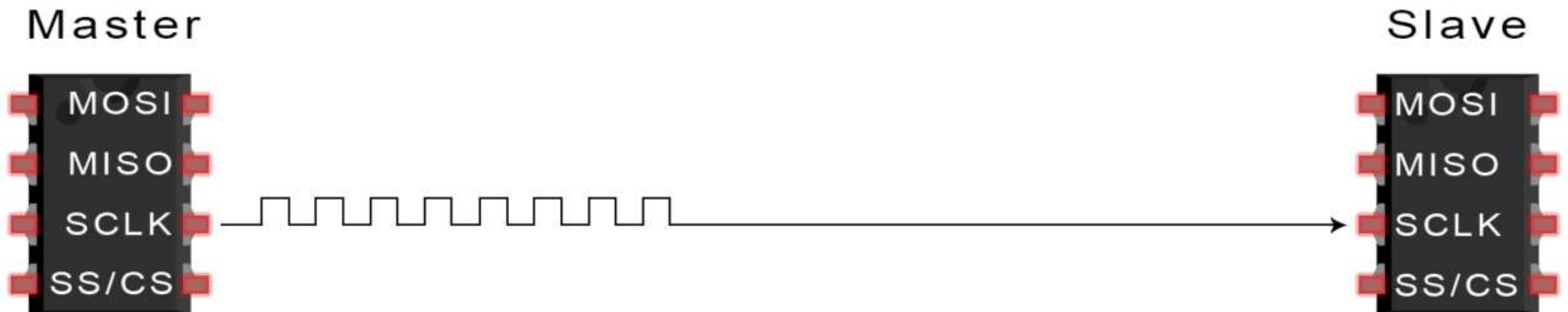
Source: <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>

Daisy Chain Configuration

SERIAL PERIPHERAL INTERFACE

STEPS OF SPI COMMUNICATION

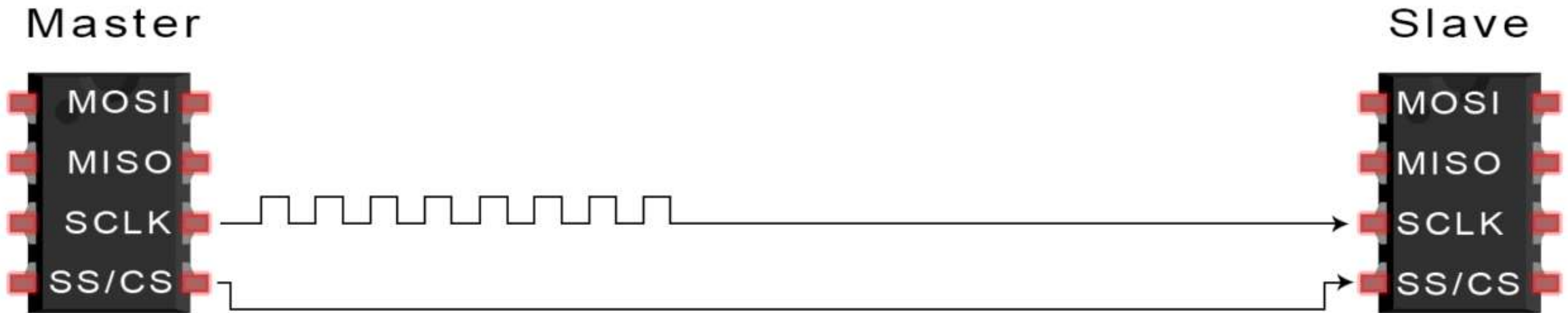
- ❑ **STEP-1:** The master outputs the clock signal:



SERIAL PERIPHERAL INTERFACE

STEPS OF SPI COMMUNICATION

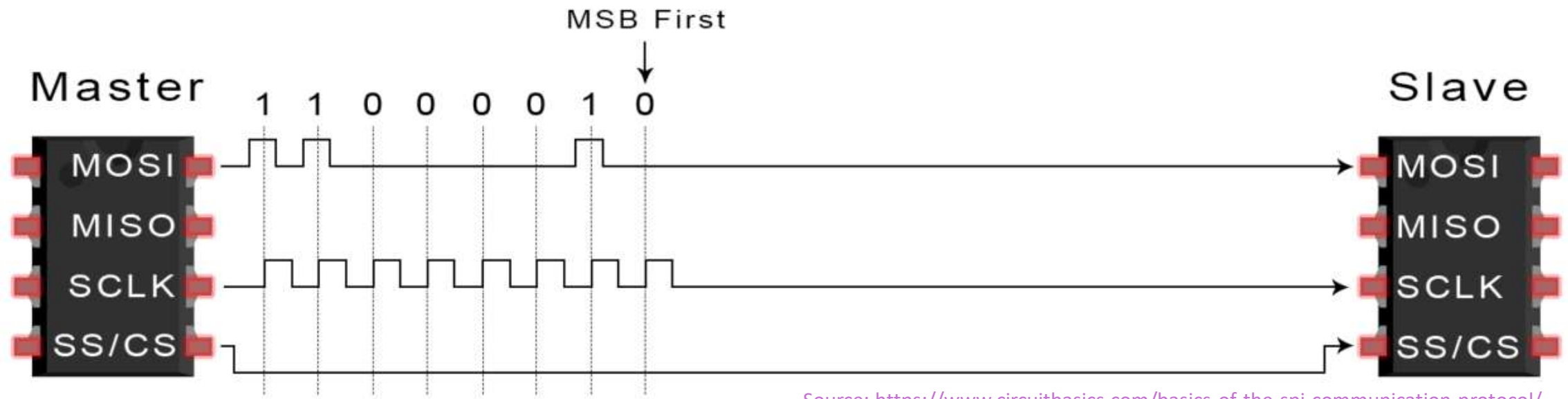
- ❑ **STEP-2:** The master configures the SS/CS pin to a low voltage state to activate the slave:



SERIAL PERIPHERAL INTERFACE

STEPS OF SPI COMMUNICATION

- ❑ **STEP-3:** In MOSI pin master sends data serially one bit at a time to slave and slave receives the data in receiving sequence

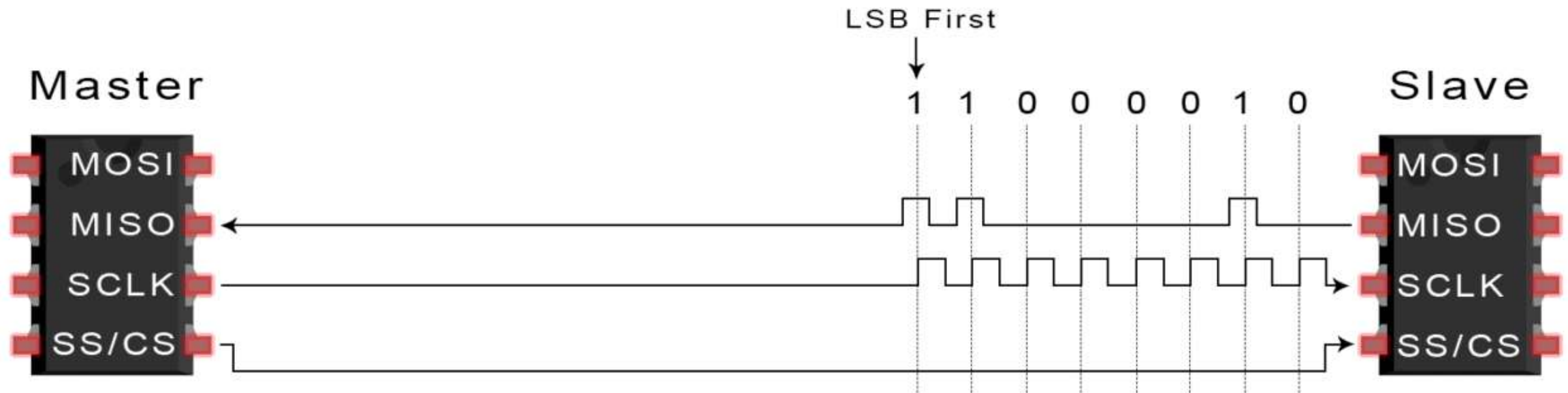


Source: <https://www.circuitbasics.com/basics-of-the-spi-communication-protocol/>

SERIAL PERIPHERAL INTERFACE

STEPS OF SPI COMMUNICATION

- ❑ **STEP-4:** Slave responds to the master by sending one bit at a time and master reads as the message received



SERIAL PERIPHERAL INTERFACE

MODES OF OPERATION

- ❑ The clock logic regulates the sending of data and it can be configured in 4 possible modes

Mode	Clock Polarity	Clock Phase
SPI_Mode0	0	0
SPI_Mode1	0	1
SPI_Mode2	1	0
SPI_Mode3	1	1

- ❑ Clock polarity refers to the logic at which the clock is set when no data is being transferred.
- ❑ The Clock phase represents whether the data will be read by the devices on the rising or falling edges of the clock pulses.

SERIAL PERIPHERAL INTERFACE

APPLICATIONS

❑ Applications of SPI protocol:

- Memory: SD Card , MMC , EEPROM , Flash
- Sensors: All kind of sensors
- Control Devices: ADC , Trim POTS and Audio Codec.
- Others: Camera Lens Mount, touchscreen, LCD and so on

SERIAL PERIPHERAL INTERFACE

ADVANTAGES

- Full duplex **communication**.
- High-speed of data communication
- Not limited to 8-bits **while transferring**
- **Slave** uses a master clock instead of **oscillators**.
- **Master devices handles all the Slaves, hence no chance of conflict**
- **Continuous streaming of data without start and stop bits**
- **Simple slave addressing system**

SERIAL PERIPHERAL INTERFACE

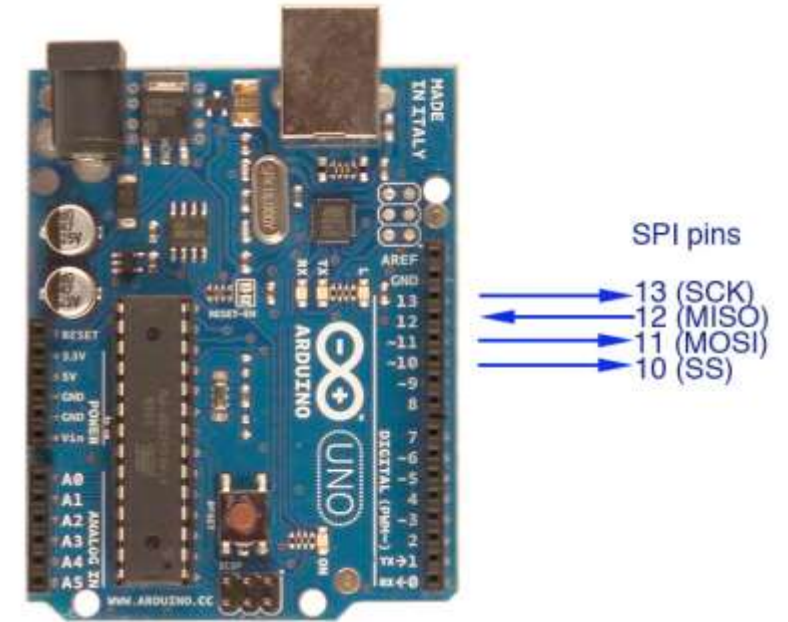
DISADVANTAGES

- Uses four pins
- Only single master is allowed in SPI
- Dedicated pin is required for slave on master (CS/SS)
- Acknowledgement mechanism is not provided
- Device based speed of data transmission

SERIAL PERIPHERAL INTERFACE

Arduino function for SPI protocol

- SPI.begin() - Initialize the SPI bus
- SPI.end() – Disables the SPI bus
- SPI.beginTransaction() – Initialize SPI bus using SPISettings
 - ex: `SPI.beginTransaction(SPISettings(14000000, MSBFIRST, SPI_MODE0))`
- SPI.endTransaction() – Stop using SPI bus
- SPI.setClockDivider(divider) – Set the SPI clock divider
 - (divider - 2, 4, 8, 16, 32, 64 or 128)
- SPI.setDataMode(mode) – Set the data mode
 - (Mode – Mode_0, Mode_1, Mode_2, Mode_3)
- SPI.transfer(val) - the byte data to be transferred (send and receive) over the bus
- SPI.transfer(buffer, size) – the array of data to be transferred (send and receive)
- SPI.useInterrupt(interruptNumber) – Used for registering interrupt number



Refer. URL : <https://www.arduino.cc/en/reference/SPI>

SERIAL PERIPHERAL INTERFACE

Arduino- Serial Peripheral Interface - Master

```
#include <SPI.h>
void setup (void) {
  Serial.begin(115200); //set baud rate to 115200 for USART
  digitalWrite(SS, HIGH); // disable Slave Select
  SPI.begin ();
  SPI.setClockDivider(SPI_CLOCK_DIV8); //divide the CPU clock by 8
}
void loop (void) {
  char c;
  digitalWrite(SS, LOW); // enable Slave Select, send test string
  for (const char * p = "Hello, world!\r" ; c = *p; p++)
  { //to send the string Hello, world!\r and use the \r as flag to signal the slave that transfer is done.
    SPI.transfer (c);
    Serial.print(c);
  }
  digitalWrite(SS, HIGH); // disable Slave Select
  delay(2000);
}
```



SERIAL PERIPHERAL INTERFACE

Arduino- Serial Peripheral Interface -Slave

```
#include <SPI.h>
char buff [50]; //stores the incoming values via SPI.
volatile byte indx; //stores the index of the 8 bit
volatile boolean process; //saves the current status of the transmission.

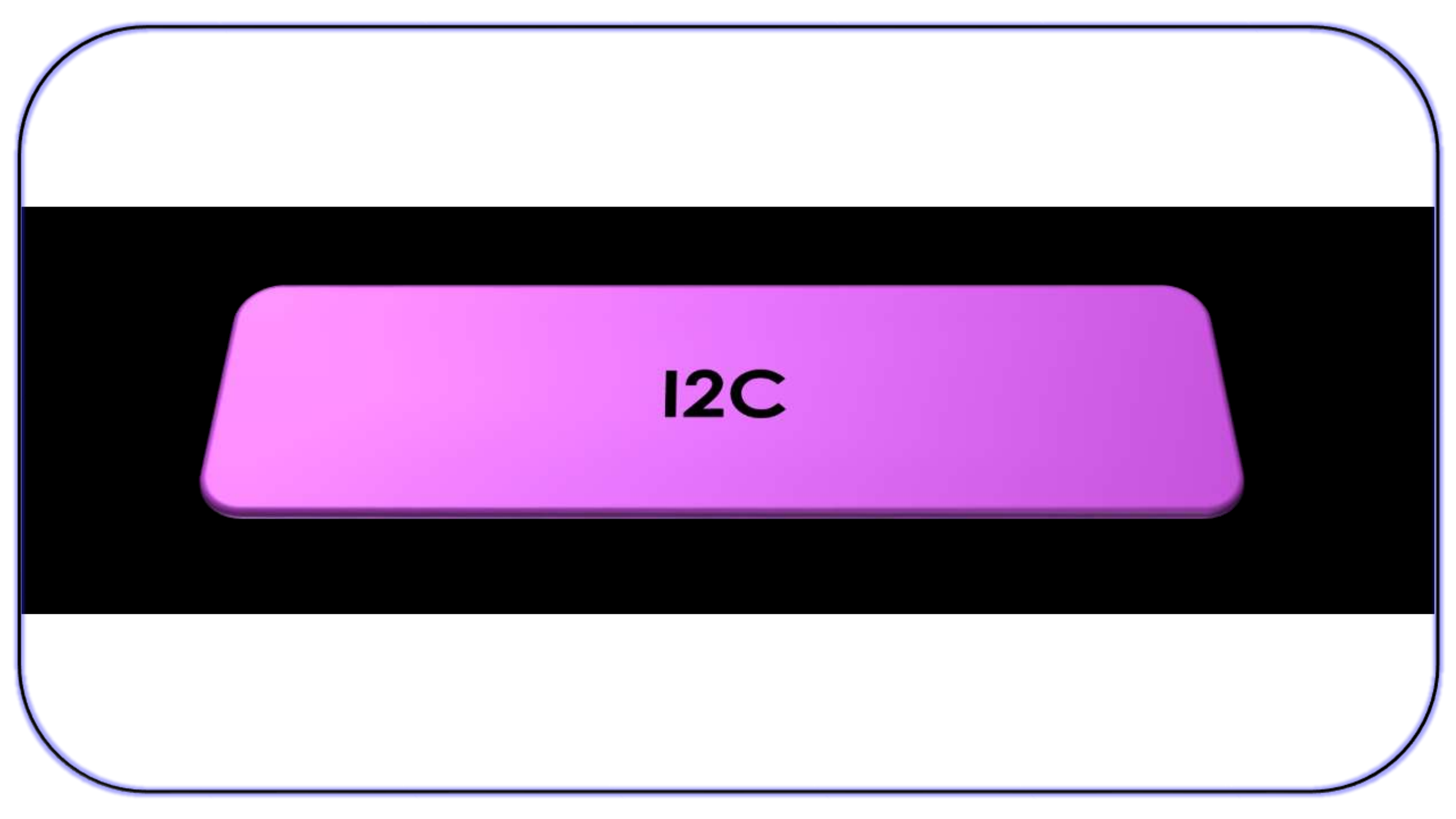
void setup (void) {
  Serial.begin (115200);
  pinMode(MISO, OUTPUT); // have to send on master in so it set as output
  SPCR |= _BV(SPE); // turn on SPI in slave mode (SPCR - SPI control register – 8 bit register)
  indx = 0; // buffer empty SPE – enables SPI when 1
  process = false;
  SPI.attachInterrupt(); // turn on interrupt
}
ISR (SPI_STC_vect) // SPI interrupt routine
{
  byte c = SPDR; // read byte from SPI Data Register
```

SERIAL PERIPHERAL INTERFACE

Arduino- Serial Peripheral Interface -Slave

```
if (indx < sizeof buff) {  
    buff [indx++] = c; // save data in the next index in the array buff  
    if (c == '\r') //check for the end of the word  
        process = true;  
}  
}  
  
void loop (void) {  
    if (process) {  
        process = false; //reset the process  
        Serial.println (buff); //print the array on serial monitor  
        indx= 0; //reset button to zero  
    }  
}
```

Refer. URL : <https://diyi0t.com/spi-tutorial-for-arduino-and-esp8266/>

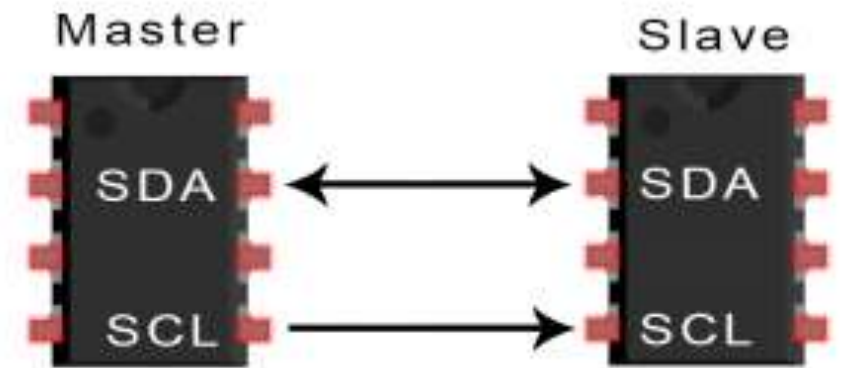


I2C

INTER-INTEGRATED CIRCUIT

INTRODUCTION

- ❑ I2C is invented in 1989 by Philips semiconductor
- ❑ It is a serial protocol with only two wire interface like UART
- ❑ Widely used for short distance and intra-board communication
- ❑ I2C has the best features of both SPI and UART protocols
- ❑ It can support multi-master and multi-slave configuration



INTER-INTEGRATED CIRCUIT

INTRODUCTION

- ❑ I2C protocol use only two pins Serial Data (SDA) and Serial Clock (SCL)
- ❑ Data is transmitted bit by bit along SDA in I2C
- ❑ **SDA (Serial Data)** – Bidirectional connection between master and slave
- ❑ **SCL (Serial Clock)** – clock signal

INTER-INTEGRATED CIRCUIT

INTRODUCTION

- ❑ I2C is also a synchronous protocol, hence the output is synchronized using clock signal
- ❑ Common clock signal is sharing between master and slave
- ❑ Master always controls the clock signal
- ❑ Master device addresses the slave devices via SCL

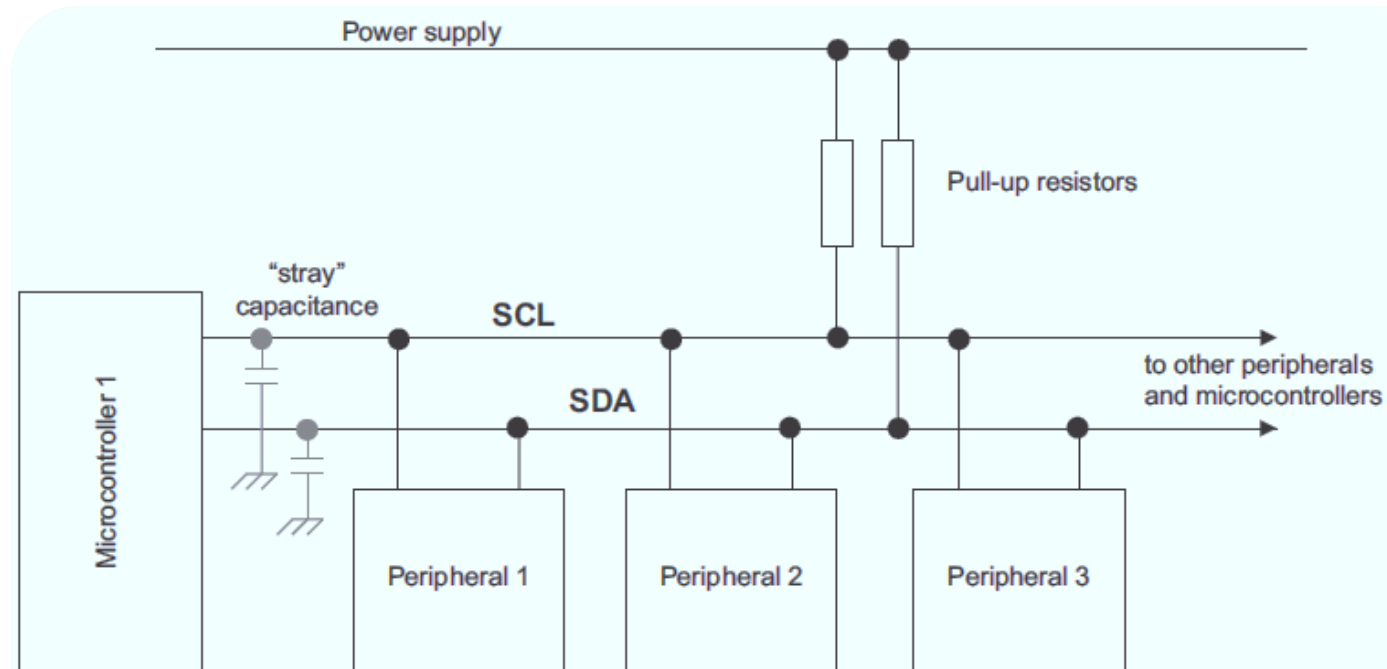
INTER-INTEGRATED CIRCUIT

HOW I2C WORKS

- ❑ Data is transferred as messages in I2C
- ❑ Messages are broken to frames of data
- ❑ Each message contains a binary address frame of the slave and data frames
- ❑ One or many data frames with data can be transferred in one message
- ❑ The message has start and stop conditions, read, write, ACK & NACK bit between each data frame

INTER-INTEGRATED CIRCUIT

INTRODUCTION

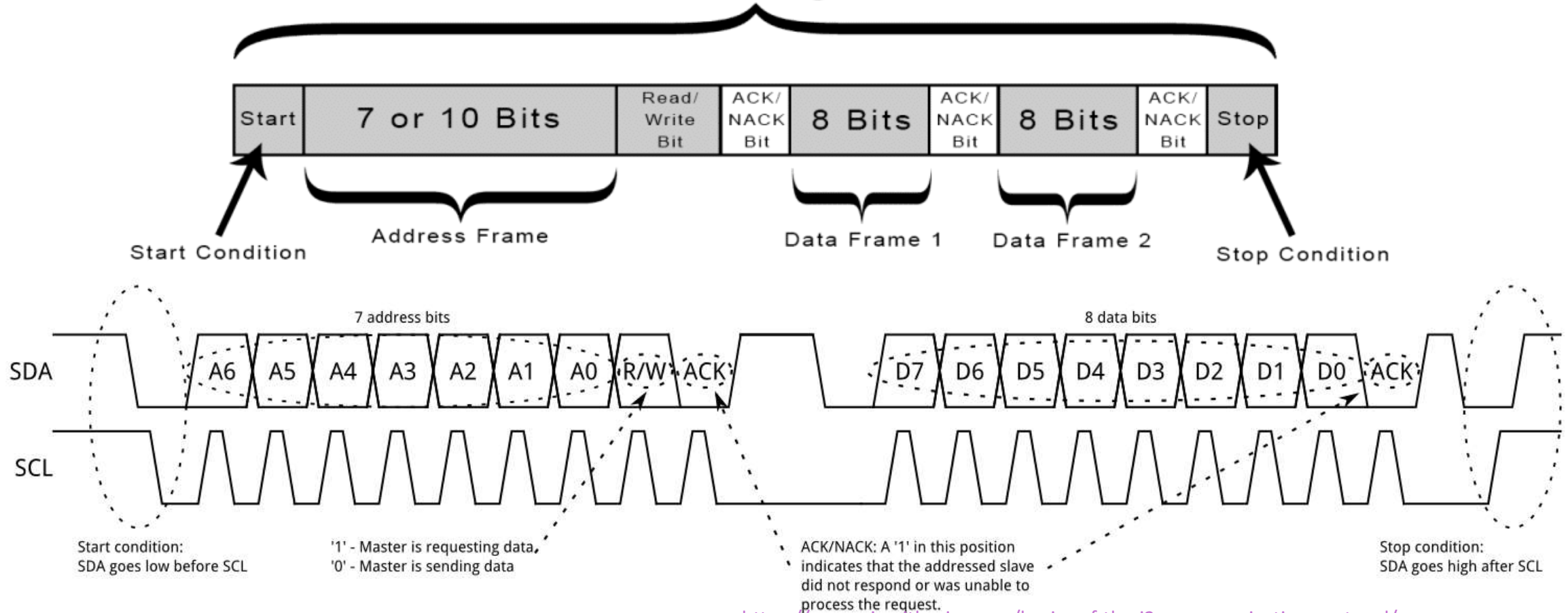


An inter-integrated circuit-based system.

INTER-INTEGRATED CIRCUIT

HOW I2C WORKS

Message



INTER-INTEGRATED CIRCUIT

HOW I2C WORKS

- ❑ **Start Condition:** The SDA pin switches from high to low voltage level before the SCL pin switches from high to low
- ❑ **Stop Condition:** The SDA pin switches from low to high voltage after SCL pin switches from low to high
- ❑ **Address Frame:** 7 or 10 bit unique address is assigned to each slave for identifying when master wants to connect to it

INTER-INTEGRATED CIRCUIT

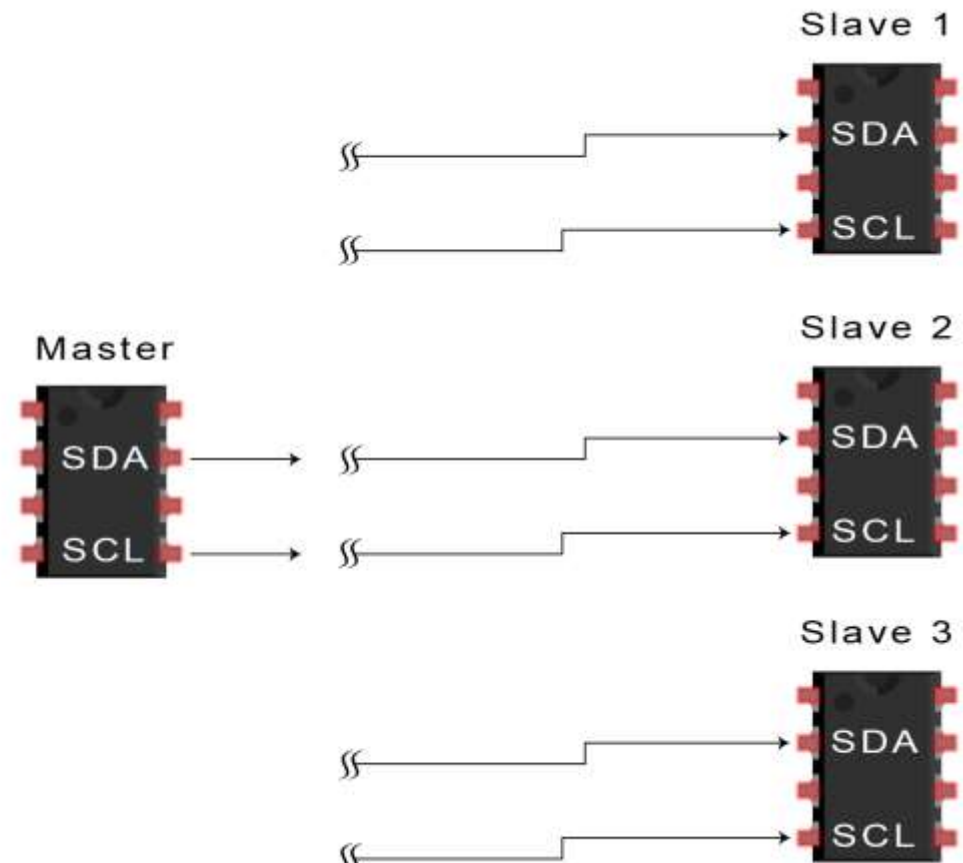
HOW I2C WORKS

- ❑ **Read/Write Bit:** This bit defines the state of master, whether it is sending data (low) or receiving data (high)
- ❑ **ACK/NACK Bit:** Each frame is followed by an ACK bit.
- ❑ **If message is successfully received, an ACK bit is returned**

INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

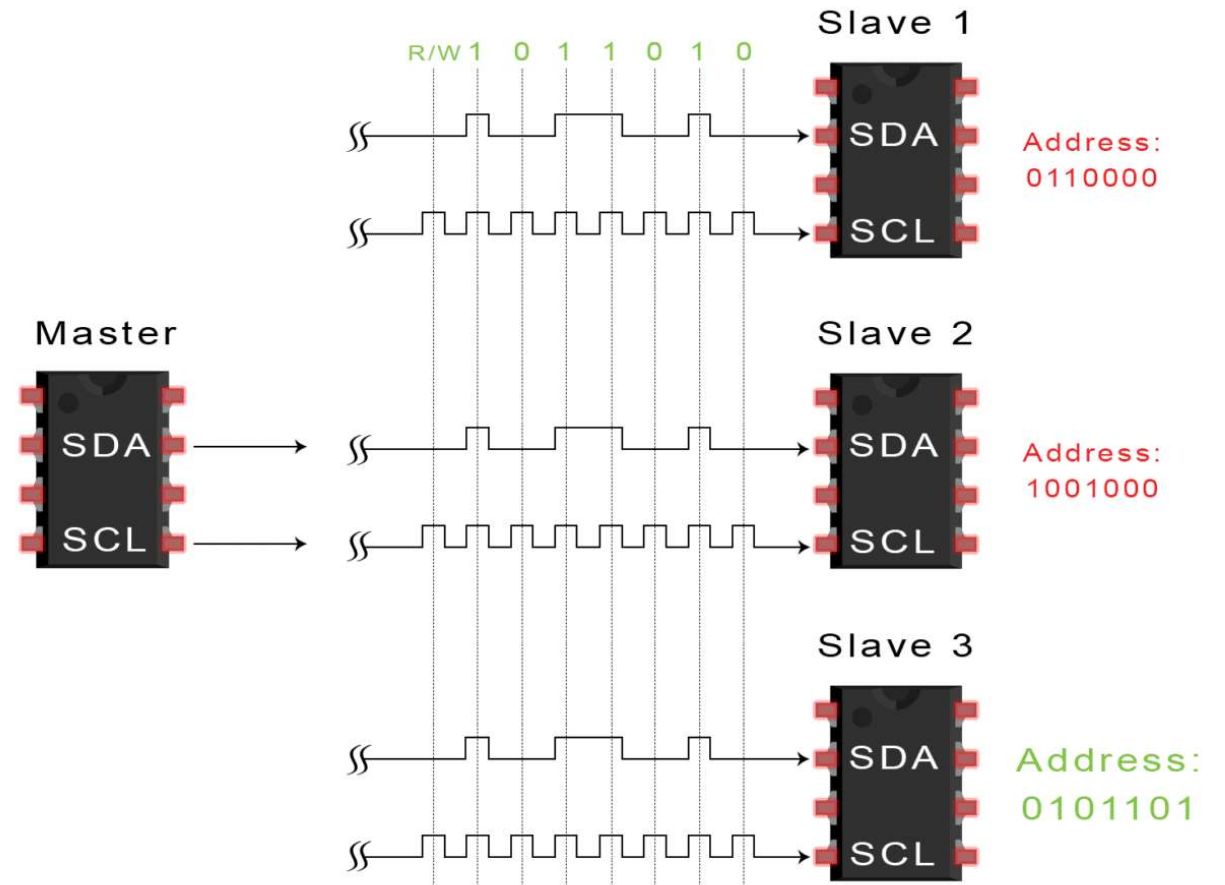
STEP-1: The master sets the start condition by setting all SDA pin to low before setting SCL pin



INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

STEP-2: The master sends 7 or 10 bit address to each slave with which it wants to communicate with, along with the read/write bit

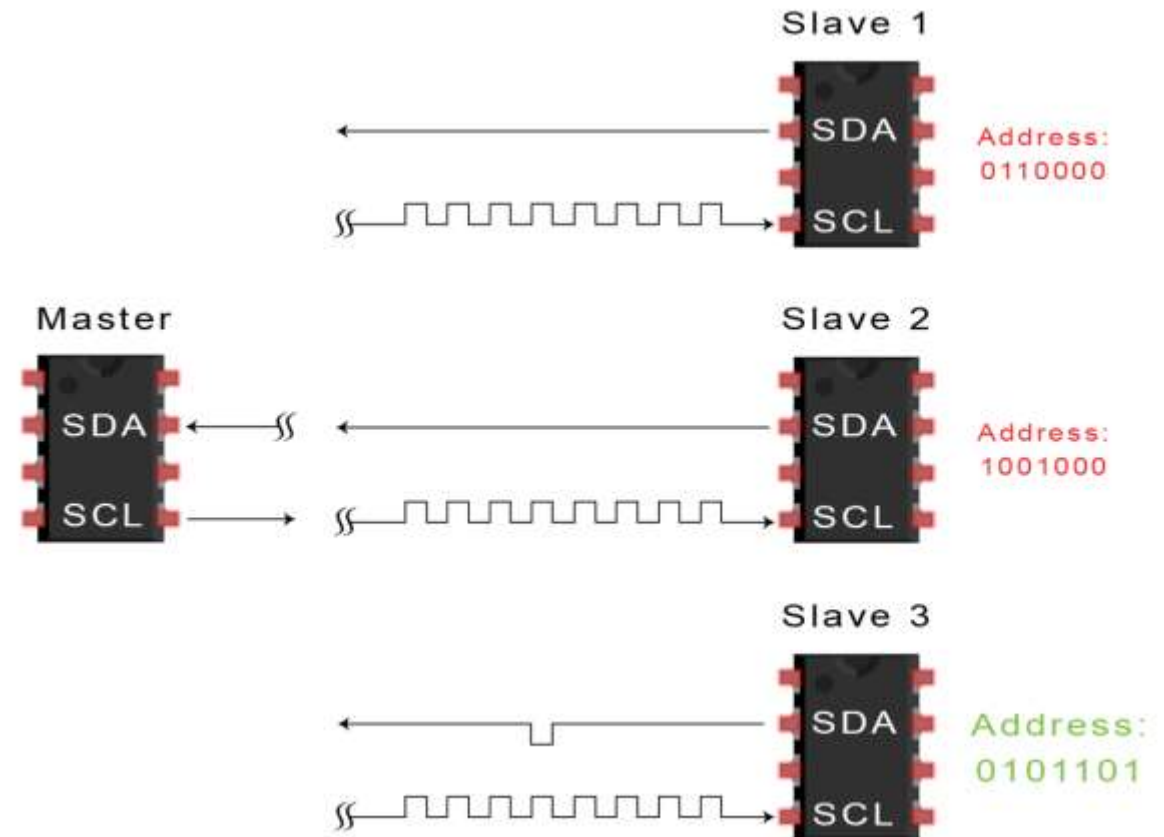


INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

STEP-3:

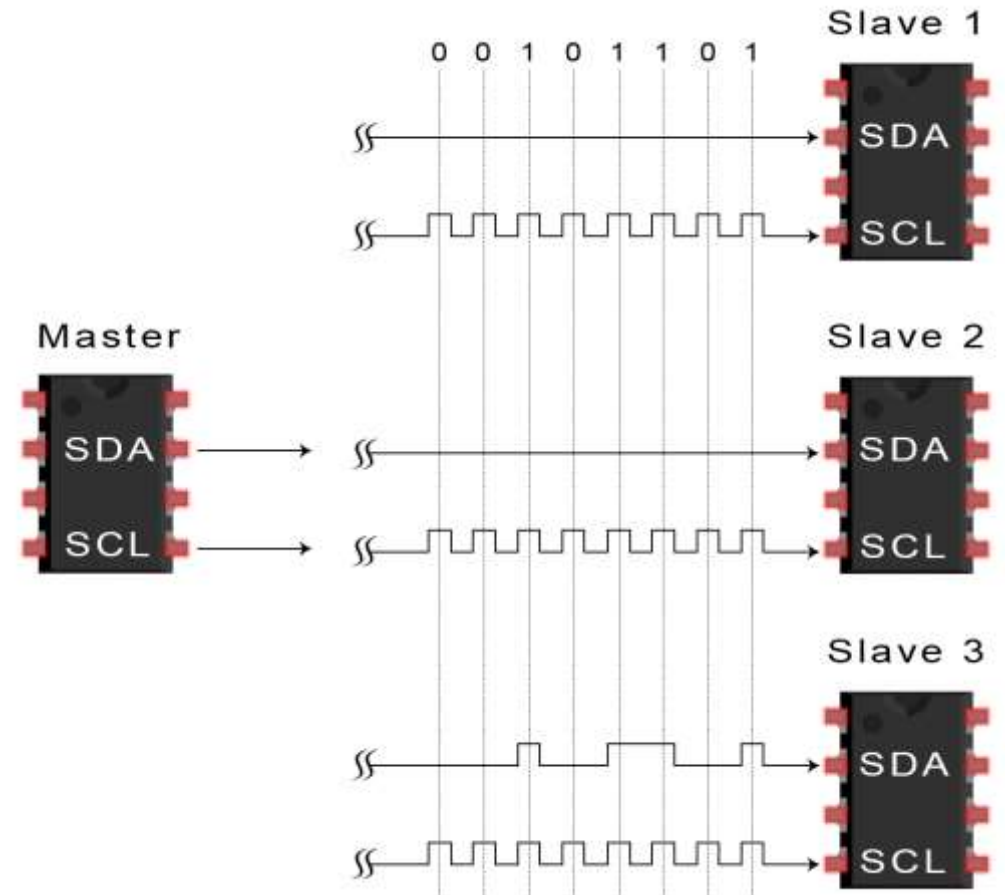
- After receiving address from master, slave compare the address with its own address
- If the address matches, ACK bit is sent and SDA pin is switched to low
- If address does not match, slave keeps the SDA pin high



INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

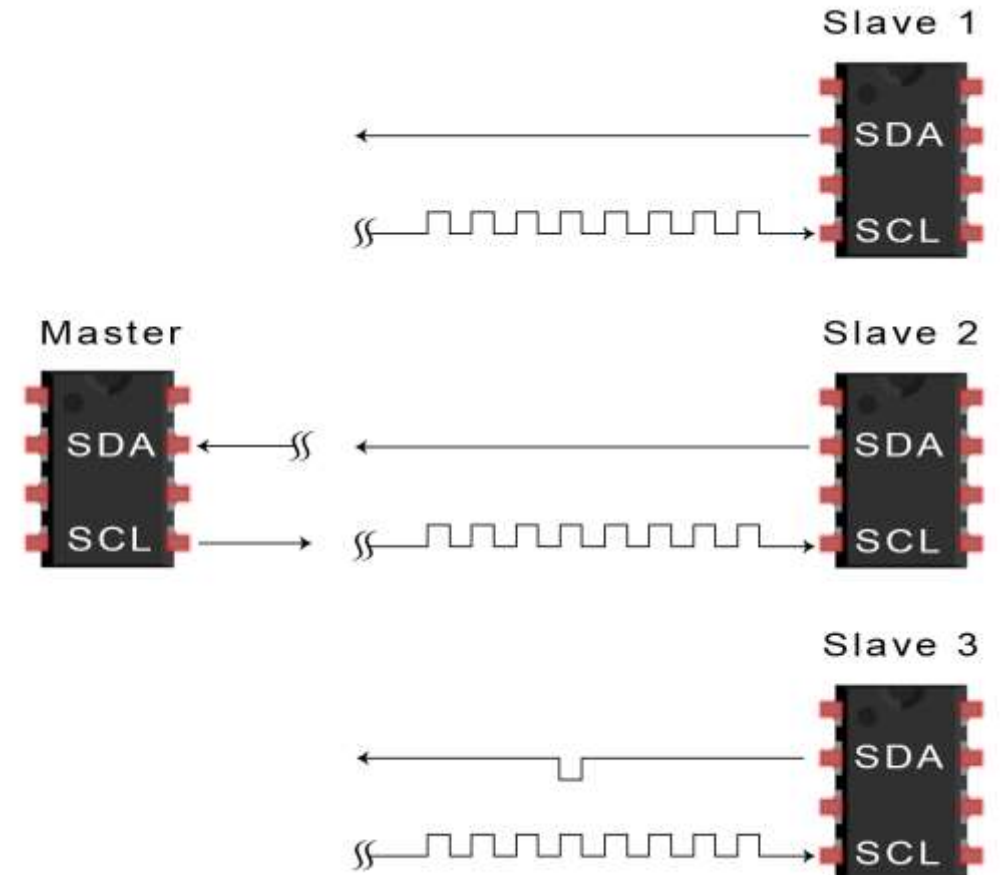
STEP-4: The master sends or receives the data frame



INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

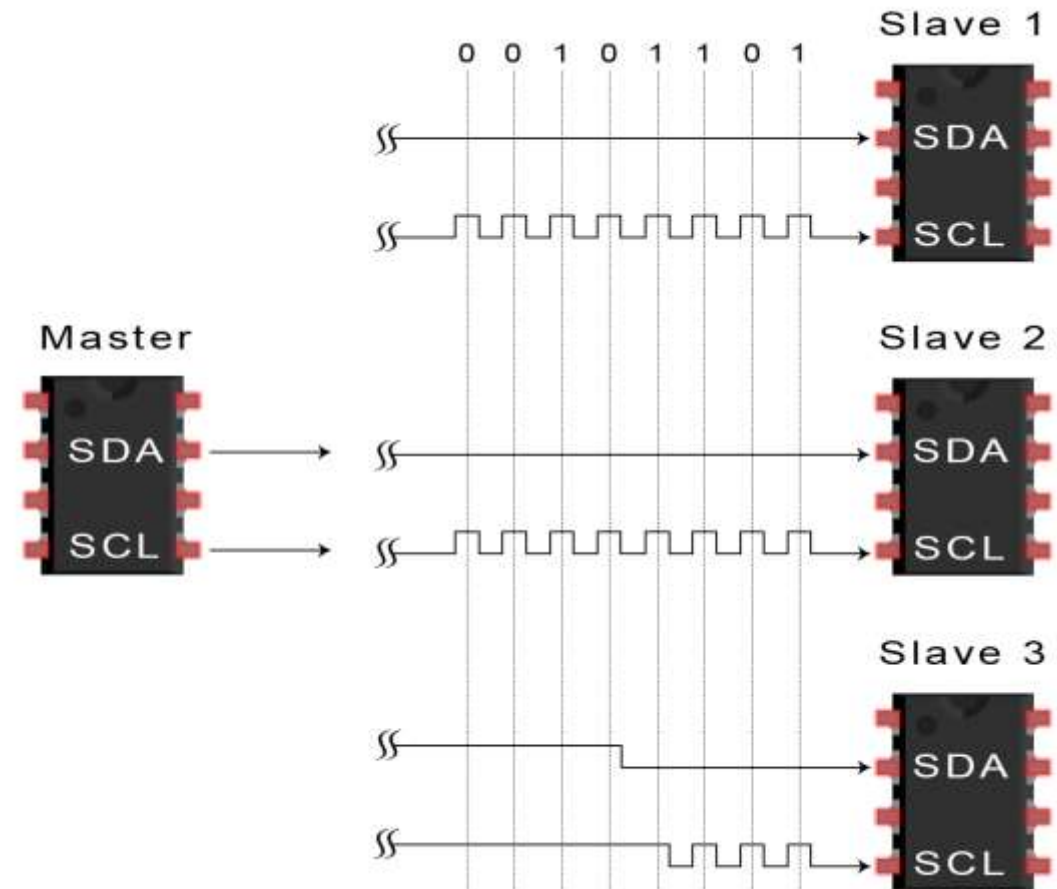
STEP-5: After each successful data reception, the receiving device sent an ACK bit



INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

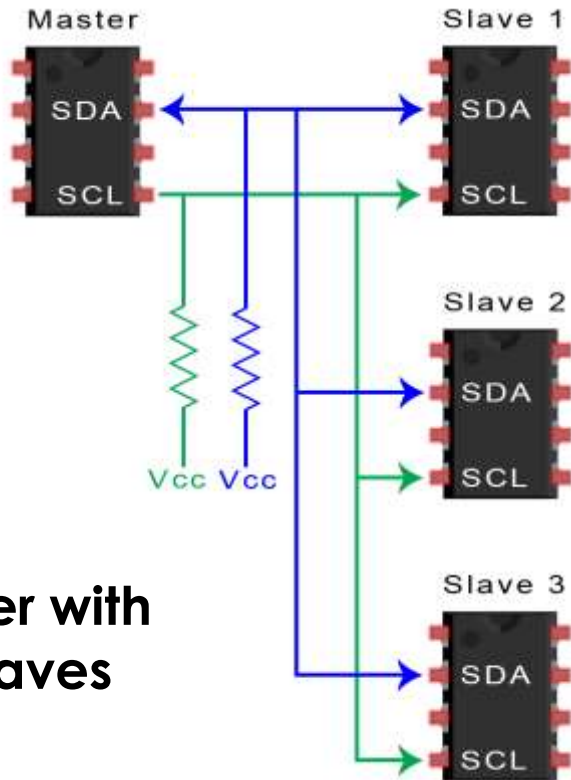
STEP-6: To end the data transmission, the master sets the SCL pin high before setting SDA pin high



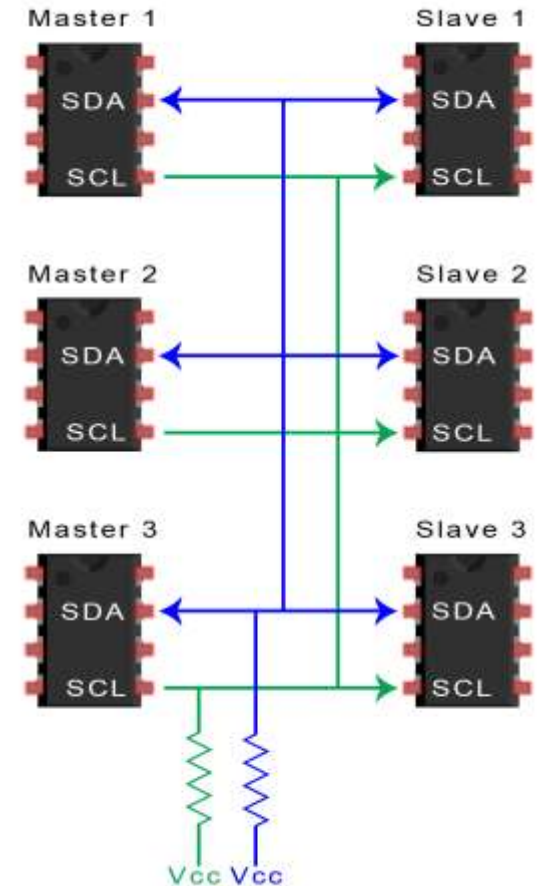
INTER-INTEGRATED CIRCUIT

STEPS OF I2C DATA TRANSMISSION

Single Master with Multiple Slaves



Multiple Master with Multiple Slaves



INTER-INTEGRATED CIRCUIT

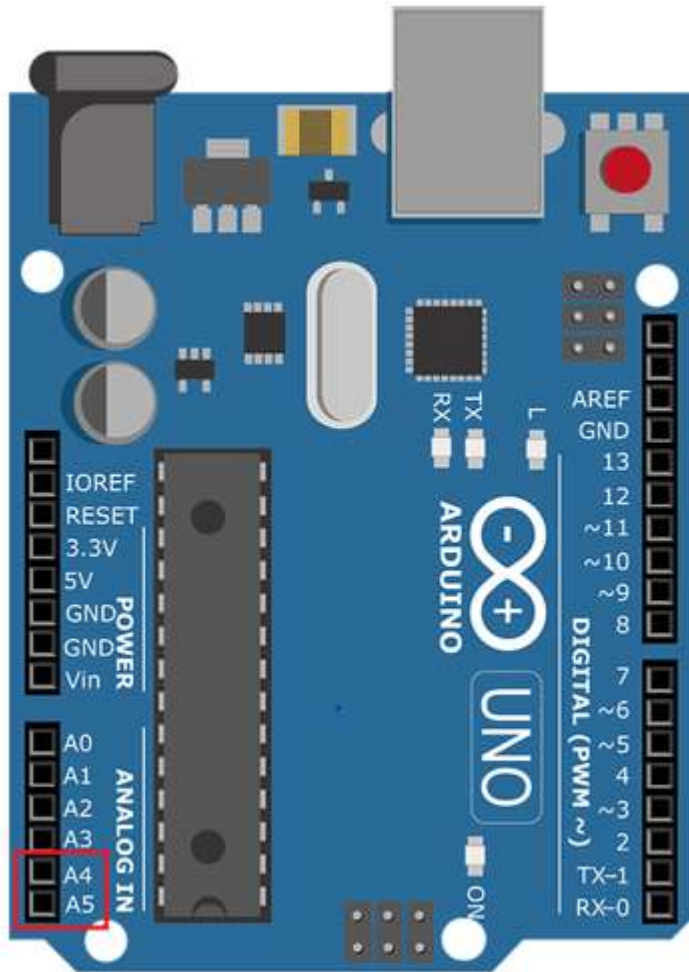
❑ ADVANTAGES:

- Only uses two wires
- Supports multiple masters and slaves
- Use chip addressing
- ACK/NACK bit for successful transmission
- it can work well with both slow and fast ICs.

❑ DISADVANTAGES:

- Slower data transfer rate than SPI
- Clock speed limitation & power dissipation because of pull-up resistor
- hardware complexity increases with multi master/slave configuration
- Frame overhead due to ack bits and device address bits

INTER-INTEGRATED CIRCUIT



I2C Line	Pin in Arduino
SDA	A4
SCL	A5

INTER-INTEGRATED CIRCUIT

Arduino function for I2C protocol (Wire Library)

Wire.begin() - Initialize the I2C bus

Wire.requestFrom(address, quantity)

address- 7 bit address of the device to request bytes

quantity- The number of bytes to request

Wire.receive() – receives a byte of data transmitted from slave to master

Wire.send() – sends data to master from slave on request

Wire.onRequest(handler) – register a function to call when master request data from slave

Wire.onReceive(handler) - Registers a function to be called when a slave device receives a transmission from a master

Wire.read() – reads a byte transmitted from slave to master

Wire.write() – writes data to slave on request from master

Wire.setClock()- modifies the clock frequency of I2C

Refer. URL : <https://www.arduino.cc/en/reference/SPI>

INTER-INTEGRATED CIRCUIT

I2C – Arduino Interface

Master read

```
include <Wire.h>
void setup()
{
  Wire.begin();      // join i2c bus (address optional for master)
  Serial.begin(9600); // start serial for output
}
void loop()
{
  Wire.requestFrom(8, 6); // request 6 bytes from slave device #8
  while (Wire.available())
  {
    // slave may send less than requested
    char c = Wire.read(); // receive a byte as character
    Serial.print(c);      // print the character
  }
  delay(500);
}
```

INTER-INTEGRATED CIRCUIT

I2C – Arduino Interface

Slave sender

```
#include <Wire.h>
void setup()
{
  Wire.begin(8);          // join i2c bus with address #8
  Wire.onRequest(requestEvent); // register event
}
void loop()
{
  delay(100);
}
// function that executes whenever data is requested by master
// this function is registered as an event, see setup()
void requestEvent()
{ Wire.write("hello "); // respond with message of 6 bytes // as expected by
  master
}
```

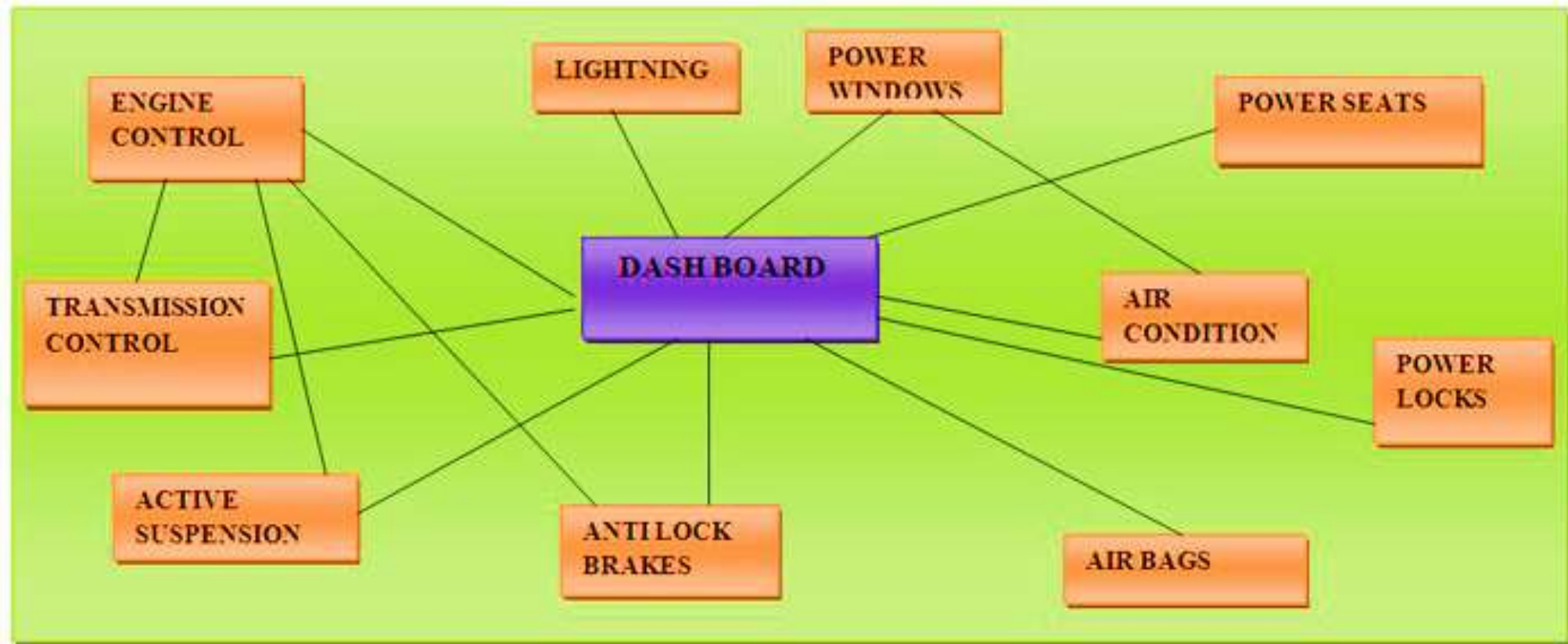
CONTROL AREA NETWORK

INTRODUCTION

- ❑ CAN bus allows all ECUs to communicate with each other via a single bus
- ❑ CAN bus has two wires- CAN low and CAN high
- ❑ **NEED FOR CAN:**
 - Simple and low cost – Communicate via one single bus
 - Fully centralized – one point of entry
 - Extremely robust – towards electromagnetic interference and electric disturbance
 - Efficient – CAN frame has ID for prioritization

CONTROL AREA NETWORK

INTRODUCTION



Pictorial view of the point-to-point wiring connection

CONTROL AREA NETWORK

INTRODUCTION

- ❑ CAN protocol is a set of rules for transmitting and receiving messages in a network of electronic devices.
- ❑ It is a method of communication used in automobiles to communicate between electronic devices like active suspension, ABS, gear control, lighting control, air conditioning, airbags, central locking etc.
- ❑ Every electronic device is connected using a common serial bus to receive and transmit data using CAN protocol

CONTROL AREA NETWORK

INTRODUCTION

- ❑ The node must have hardware and software embedded in them to have data exchange
- ❑ **Host controller** (ECU/MCU) is responsible for the functioning of the respective node.
- ❑ **CAN controller** convert the messages in accordance with the CAN protocols for transmitting via CAN transceiver
- ❑ CAN controller can either be connected separately or embedded inside the host controller

CONTROL AREA NETWORK

ADVANTAGES

- **Low cost:** They are cheap on bulk production
- **Reliable:** Highly immune to EMI and excellent error detection and error handling
- **Flexibility:** can be easily connected / disconnected
- **Good Speed:** 1 MBit/s @ 40m bus length.
- **Multi-master communication**
- **Fault Confinement**
- **Broadcast capability**

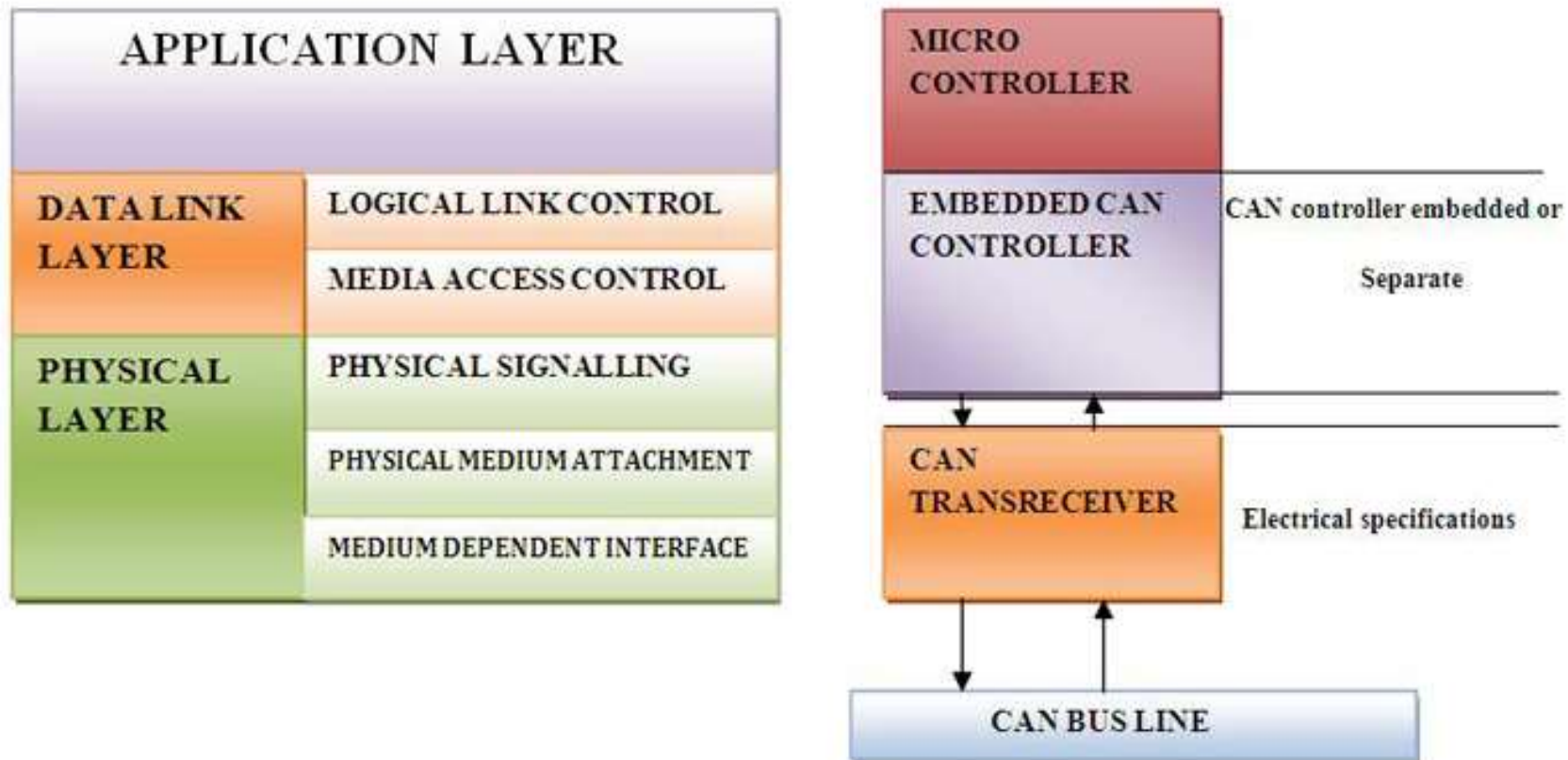
CONTROL AREA NETWORK

CAN ARCHITECTURE

- ❑ CAN uses the existing OSI reference model for data transmission
- ❑ CAN protocol uses lower two layers of OSI **physical layer and data link layer**.
- ❑ The System designer defined the functionality of remaining five layers according to needs
- ❑ **Host controller** uses application layer of OSI model
- ❑ CAN controller incorporate **logical link control (LLC)** and **MAC medium access control** of data link layer.
- ❑ LLC allows filtering of messages by using **unique ID**

CONTROL AREA NETWORK

CAN ARCHITECTURE



CONTROL AREA NETWORK

CAN ARCHITECTURE

- ❑ Once, framing is done it is followed by arbitration, error detection and acknowledgement handled by MAC layer
- ❑ CAN trans-receiver takes care of encoding and decoding.
- ❑ Finally CAN trans-receiver synchronizes with the CAN bus for message transmission

CONTROL AREA NETWORK

HOW CAN PROTOCOL WORKS?

- ❑ **Master-slave** configuration is not supported in the CAN protocol architecture
- ❑ Each node cannot access every other node for reading and writing data on CAN bus
- ❑ Based on the availability of bus, CAN bus writes the CAN frame
- ❑ A frame is a defined structure, carrying sequence of bit or bytes of data
- ❑ In CAN protocol, predefined unique ID is used instead of destination address

CONTROL AREA NETWORK

HOW CAN PROTOCOL WORKS?

- ❑ Every byte of information is defined in the CAN protocol
- ❑ All nodes receive the CAN frame and based on the ID nodes decides whether to accept it or not
- ❑ Highest priority message gets the bus access and low priority nodes will wait till the bus is available
- ❑ Nodes can be added without re-programming

CONTROL AREA NETWORK

MESSAGE FRAMING

- ❑ Messages in CAN are sent in a format called frames. Framing of message is done by **MAC sub layer** of Data Link Layer.
- ❑ There are two type of frames **standard or extended**. These frames can be differentiated on the basis of **identifier fields**.
- ❑ A CAN frame with **11 bit identifier fields** called Standard CAN frame and with **29 bit identifier field** is called extended CAN frame
- ❑ Binary values in CAN protocol are termed as **dominant (logic 0)** and **recessive (logic 1)** bits.

CONTROL AREA NETWORK

STANDARD CAN - MESSAGE FRAMING

SOF	11 bit identifier	RTR	IDE	r0	DLC	0...8 byte data	CRC	ACK	EOF	IFS
-----	----------------------	-----	-----	----	-----	-----------------------	-----	-----	-----	-----

Various fields in standard CAN

CONTROL AREA NETWORK

STANDARD CAN - MESSAGE FRAMING

- ❑ **SOF - Start of Frame bit.** It indicates start of message and used to synchronize the nodes on a bus. A dominant bit (logic 0) in the field marks the start of frame.
- ❑ **IDENTIFIER-**It serves dual purpose, one to determine which node has access to the bus and second to identify the type of message.
- ❑ **RTR - Remote Transmission Request.** It identifies whether it's a data frame or a remote frame. RTR is dominant when it is a data frame and recessive when it is a remote frame

CONTROL AREA NETWORK

STANDARD CAN - MESSAGE FRAMING

- ☐ **IDE – Identifier Extension.** It is used to specify the frame format. Dominant bit is for standard and recessive for extended frame.
- ☐ **R0 - Reversed bit.** Not used currently and kept for future use.
- ☐ **DLC – Data Length Code.** It is 4 bit data length code that contains the number of bytes being transmitted.
- ☐ **DATA –** Used to store up to 64 data bits of application data to be transmitted.

CONTROL AREA NETWORK

STANDARD CAN - MESSAGE FRAMING

- ❑ **CRC–** Cyclic Redundancy Check. The 16-bit (15 bits plus delimiter) cyclic redundancy check (CRC) contains the checksum of the preceding application data for error detection.
- ❑ **ACK –** Acknowledge (ACK) field. It comprises of the ACK slot (0 or 1) and the ACK delimiter (always 1). When the data is received correctly the recessive bit in ACK slot is overwritten as dominant bit by the receiver.

CONTROL AREA NETWORK

STANDARD CAN - MESSAGE FRAMING

- ❑ EOF– End of Frame (EOF). The recessive (1) 7-bits mark the end of a CAN frame (message) and disables
- ❑ IFS - Inter Frame Space that specifies minimum number of bits separating consecutive messages
- ❑ IFS provides the intermission between two frames and consists of three recessive bits known as intermission bits. This time allows nodes for internal processing before the start of next frame.

CONTROL AREA NETWORK

EXTENDED CAN - MESSAGE FRAMING

- ❑ It is same as 11-bit identifier with some added fields
- ❑ SRR- Substitute Remote Request- always transmitted as a recessive bit (1) to make sure that, standard data frame always has the high priority (0) and wins the arbitration.
- ❑ R1- It is another bit not used currently and kept for future use.

CONTROL AREA NETWORK

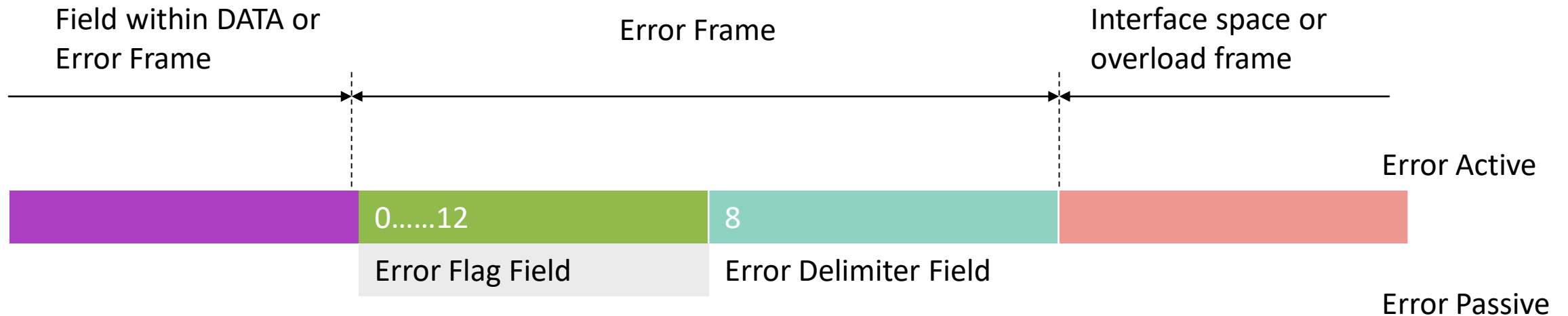
MESSAGE FRAME - TYPES

- ❑ There are four different frames which can be used on the bus.
- ❑ Data frames- most commonly used frame for data transmission from one node to another node
 - ❑ Data Frames contains Arbitration Fields, Control Fields, Data Fields, CRC Fields, a 2-bit Acknowledge Field and an End of Frame.
- ❑ Remote frames – seeks permission for data transmission from another node
 - ❑ It is similar to data frame without data field and RTR bit is recessive (1).

CONTROL AREA NETWORK

MESSAGE FRAME - TYPES

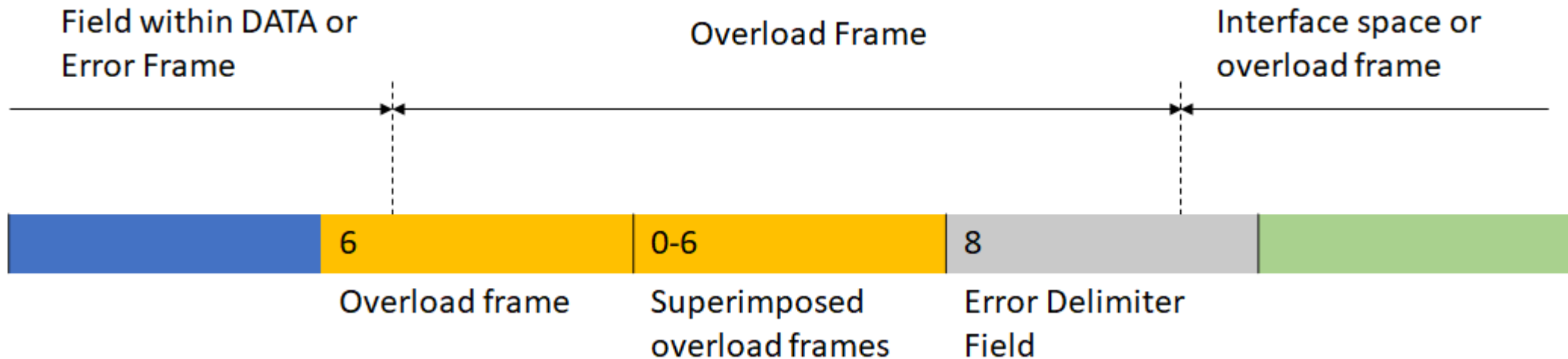
- ❑ Error frames – If transmitting or receiving node detects an error, it will immediately abort transmission and send error frame consisting of an error flag made up of dominant bits (0) and error flag delimiter made up of eight recessive bits (1).



CONTROL AREA NETWORK

MESSAGE FRAME - TYPES

- ❑ Overload frame-It is similar to error frame but used for providing extra delay between the messages. An Overload frame is generated by a node when it becomes too busy and is not ready to receive. Overload flag → dominant (0) bits and Overload delimiter → recessive (1) bits.



CONTROL AREA NETWORK

- ❑ **Future of CAN bus**
 - ❑ **Need to accommodate increasing advanced vehicle**
 - ❑ **Role of Internet of Things in vehicles**
 - ❑ **Development of autonomous vehicles**
 - ❑ **The advancement of cloud computing**
 - ❑ **To address security issues**
 - ❑ **Vehicle telematics**

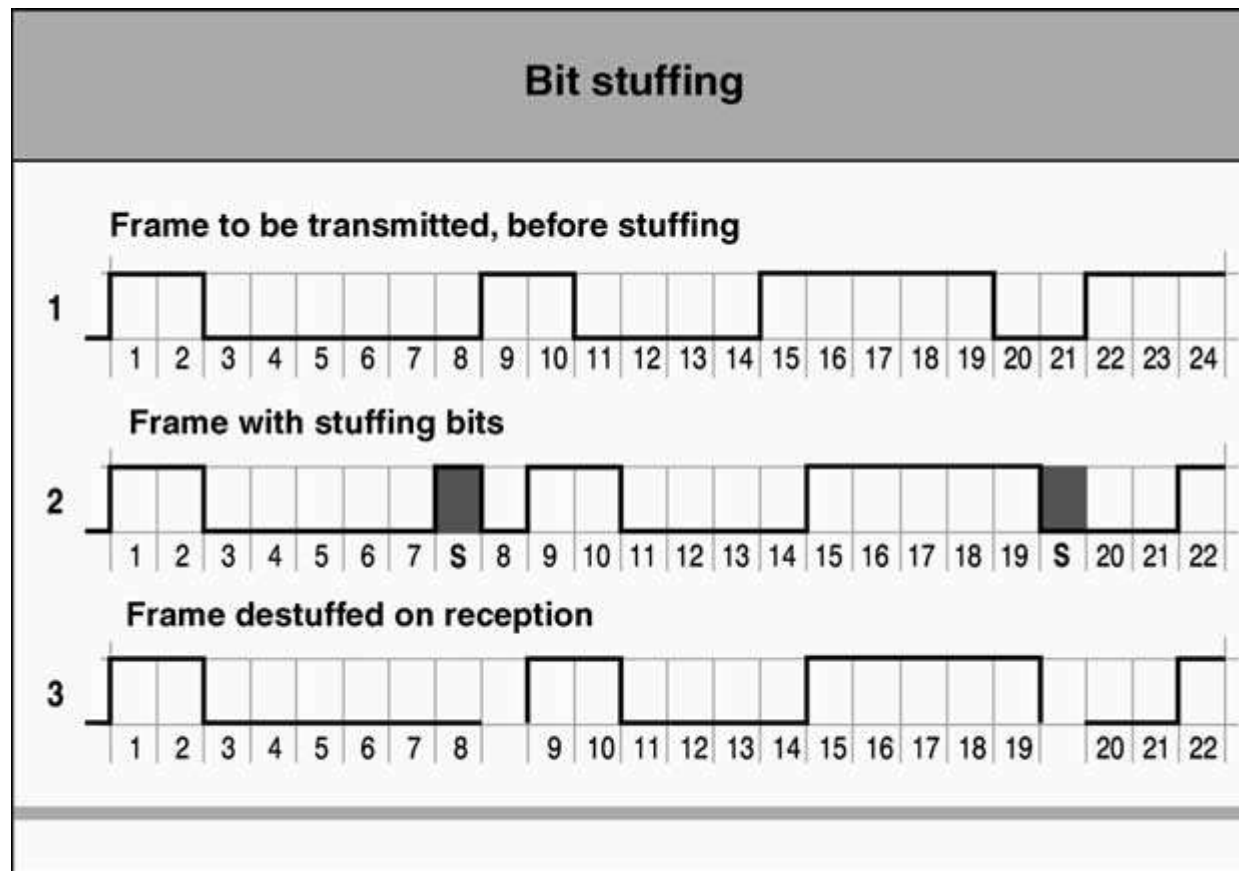
CONTROL AREA NETWORK

❑ BIT STUFFING METHOD

- ❑ After 5 bits of identical value (either dominant or recessive), a supplementary bit having an intentionally opposite value.
- ❑ To 'break the rhythm', and thus indicate that everything is all right, despite appearances.
- ❑ This technique slightly protracts the transmission time of a message.
- ❑ But it lays an active part in safeguarding its content during transport, to retain clock synchronization. Eg: 000001

CONTROL AREA NETWORK

❑ BIT STUFFING METHOD

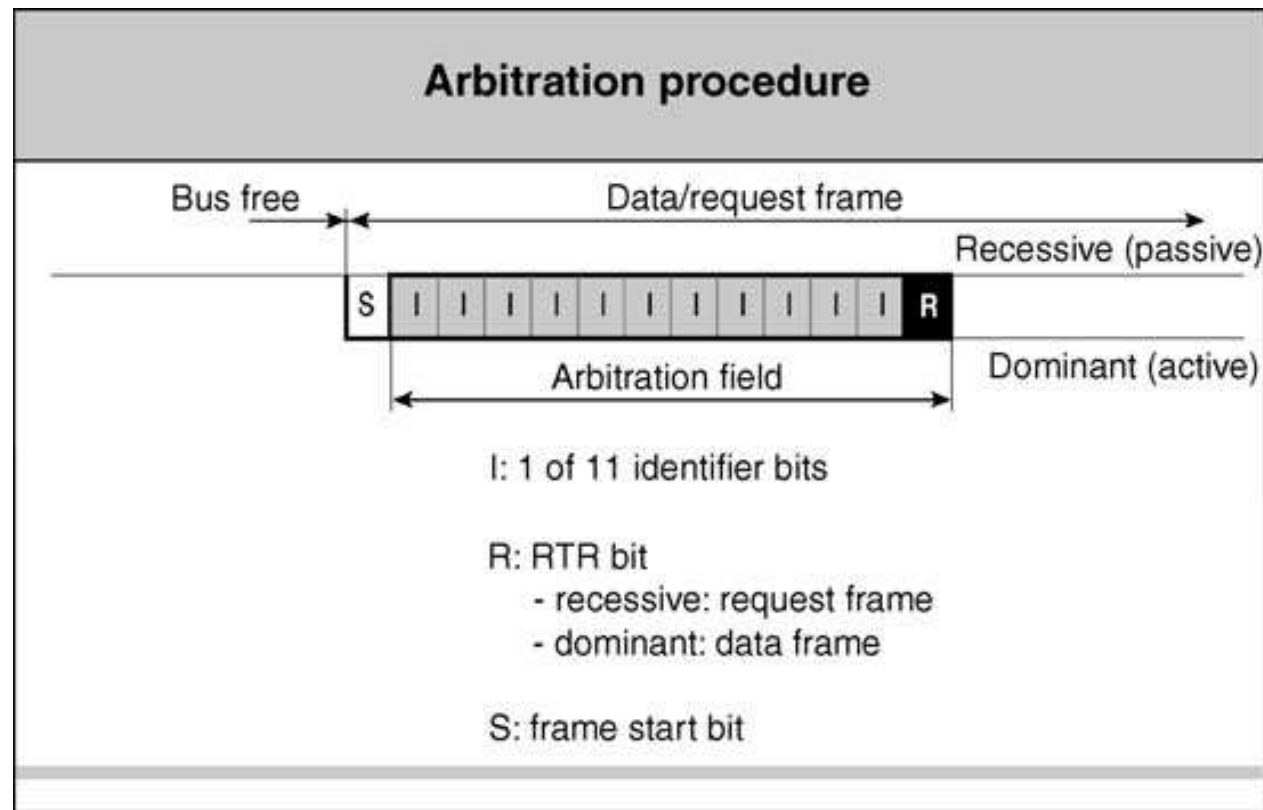


CONTROL AREA NETWORK

- ❑ **Arbitration field**
 - ❑ Set of bits in the message frame assigned to each message to control arbitration.
 - ❑ Consists of the identifier bits and the bit immediately following, called the RTR (remote transmission request).
 - ❑ **Identifier –**
 - ❑ Length of the identifier is 11 bits,
 - ❑ The bits are transmitted in the order from ID_10 to ID_0
 - ❑ the 7 most significant bits (from ID_10 to ID_4) must not all be recessive (1).
 - ❑ **RTR – Must be dominant bit (0).**

CONTROL AREA NETWORK

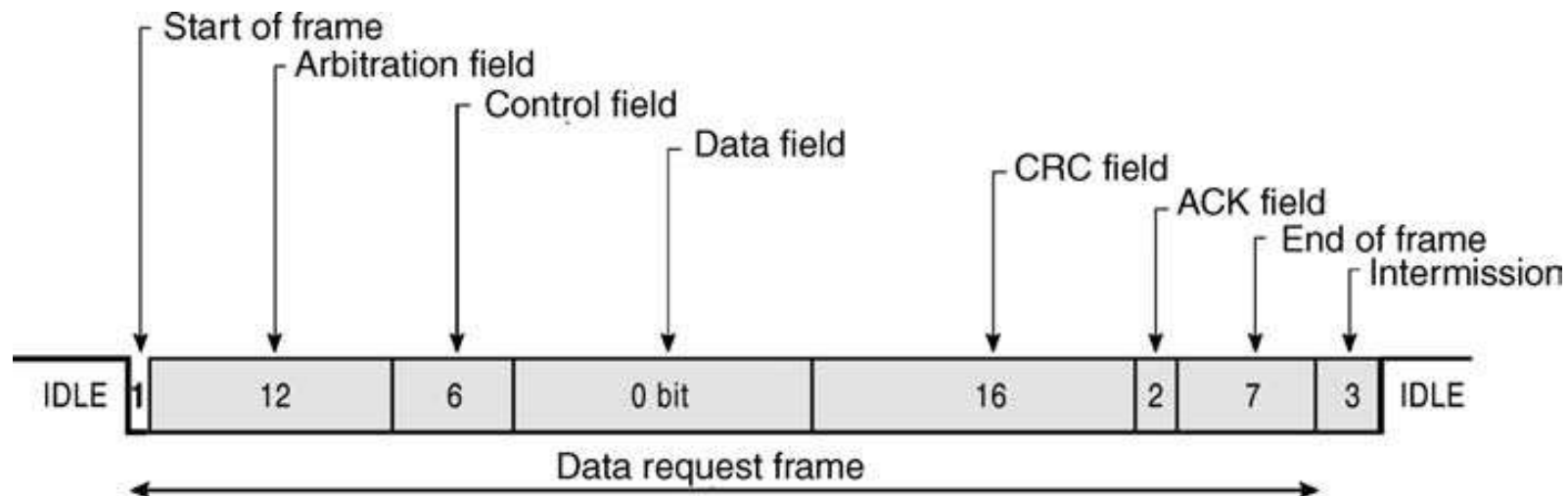
❑ Arbitration field



(a)

CONTROL AREA NETWORK

❑ Remote Frame - RTR bit – Recessive



CONTROL AREA NETWORK

Types of ERRORS in CAN

- ❑ **In Physical Layer**
 - ❑ Bit Error - bit itself is affected by errors
 - ❑ Bit Stuffing Error - involuntary (parasites, transmissions, forgotten elements) or sometimes deliberate.
- ❑ **In Frame Layer**
 - ❑ CRC delimiter error
 - ❑ ACKnowledgement delimiter error
 - ❑ end of frame error
 - ❑ error delimiter error
 - ❑ overload delimiter error
- ❑ The presence of errors will be signaled by an error frame which is generated on the bus to inform those entitled

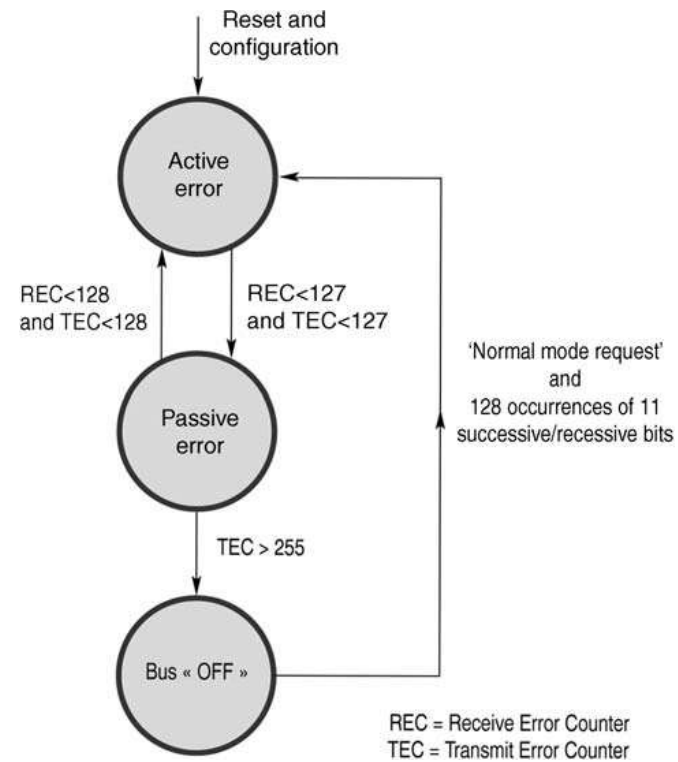
CONTROL AREA NETWORK

☐ CONFINEMENT ERRORS

- ☐ The term 'confinement' implies a vast mechanism about TEC / REC (Transmit / Receive Error Counter) having the purpose of determining whether a node is:
 - ☐ not disturbed at all,
 - ☐ slightly disturbed
 - ☐ rather more seriously disturbed by errors
 - ☐ too disturbed and must switch to bus off

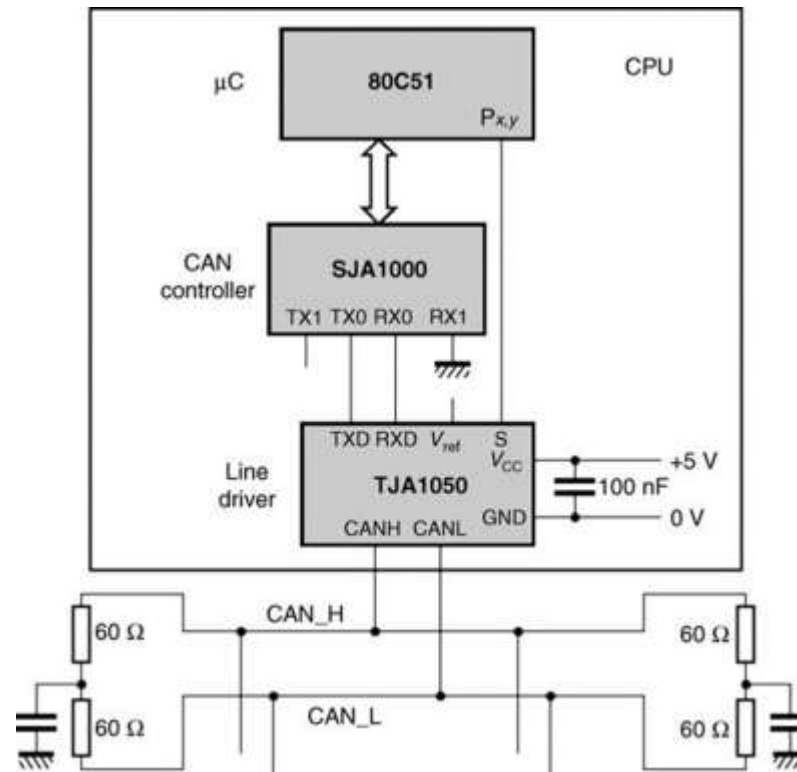
CONTROL AREA NETWORK

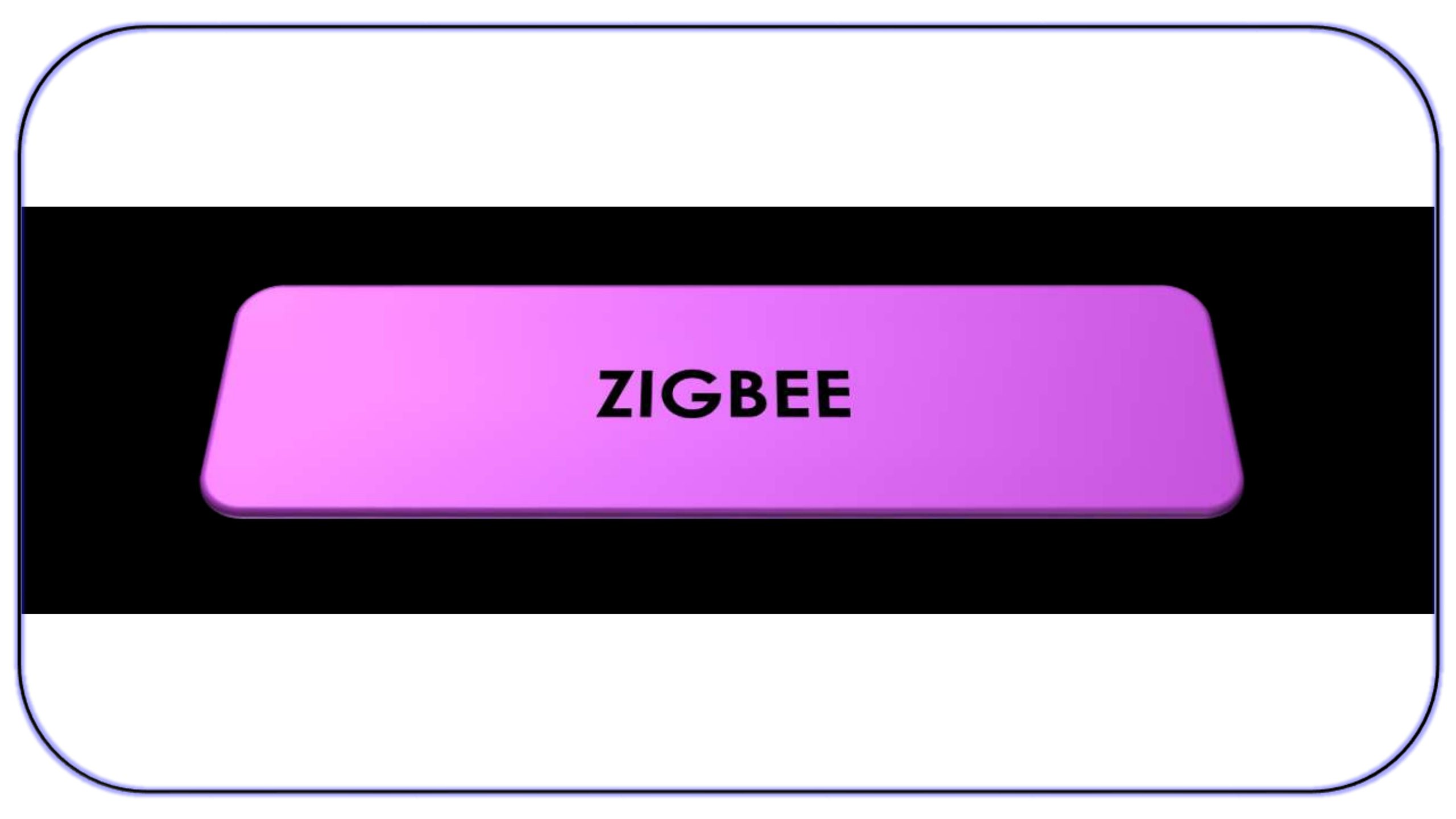
❑ Processing confinement errors



CONTROL AREA NETWORK

□ Bus Access





ZIGBEE

ZIGBEE

INTRODUCTION

- ❑ ZigBee protocol is mostly used in the power constraint short distance communication
- ❑ What other communication protocols ?
 - ❑ Bluetooth
 - ❑ Wi-Fi
- ❑ ZigBee is specially built with control and sensor applications
- ❑ ZigBee is a standard with- very low cost, low power device, low data rate and short-range communications

ZIGBEE

INTRODUCTION

- ❑ Most commonly used standard in WSN, IoT
- ❑ Open-source standard developed by ZigBee Alliance
- ❑ It works on IEEE 802.15.4 standard for wireless personal area networks
- ❑ ZigBee operates at 868 MHz (Europe), 902-928MHz (US & Australia) and 2.4 GHz (worldwide) frequencies
- ❑ Data rate of 250 kbps for low data rate applications

ZIGBEE

- ❑ ZigBee devices can cover a distance range between 10-100 meters.
- ❑ It supports master-slave configuration or master-master configuration
- ❑ ZigBee architecture consist of coordinator, router and end device
- ❑ ZigBee supports star, tree and mesh topologies
- ❑ Modes of operation – Beacon (Sync) and Beacon less (Async)

ZIGBEE

☐ Coordinator

- ☐ Root of network
- ☐ One coordinator in each network
- ☐ Performs channel selection, assigning node ID, allocation of unique address

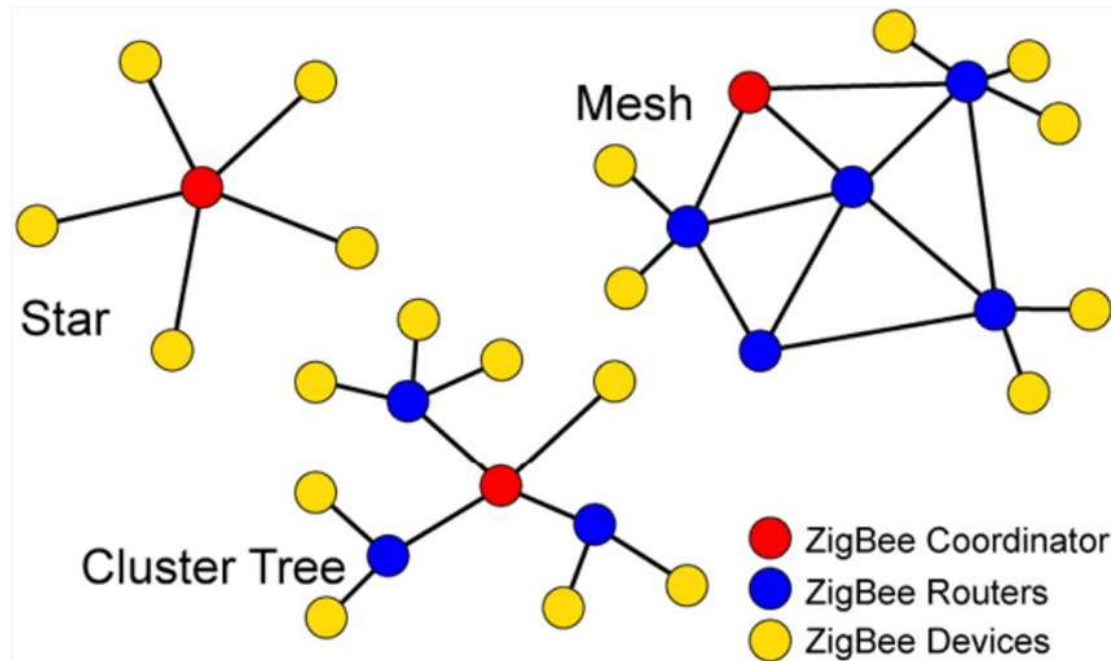
☐ Routers

- ☐ Intermediate node between coordinator and end node
- ☐ Route traffic between different nodes
- ☐ Allows other end devices to join network

ZIGBEE

❑ End Node

- ❑ sleep mode to increase battery efficiency
- ❑ All traffic to end device is first routed to parent node



ZIGBEE

- ❑ Channel Access

- ❑ The coordinator assigns only one channel to the network

- ❑ Methods of channel access

- ❑ Contention based - need not synchronized and used CSMA

- ❑ CSMA: Channel goes to receive mode and check for any signal, if no signal is detected then it will start data transmission or else wait for a random period of time

- ❑ Contention free - Allocates specific time slot to each device

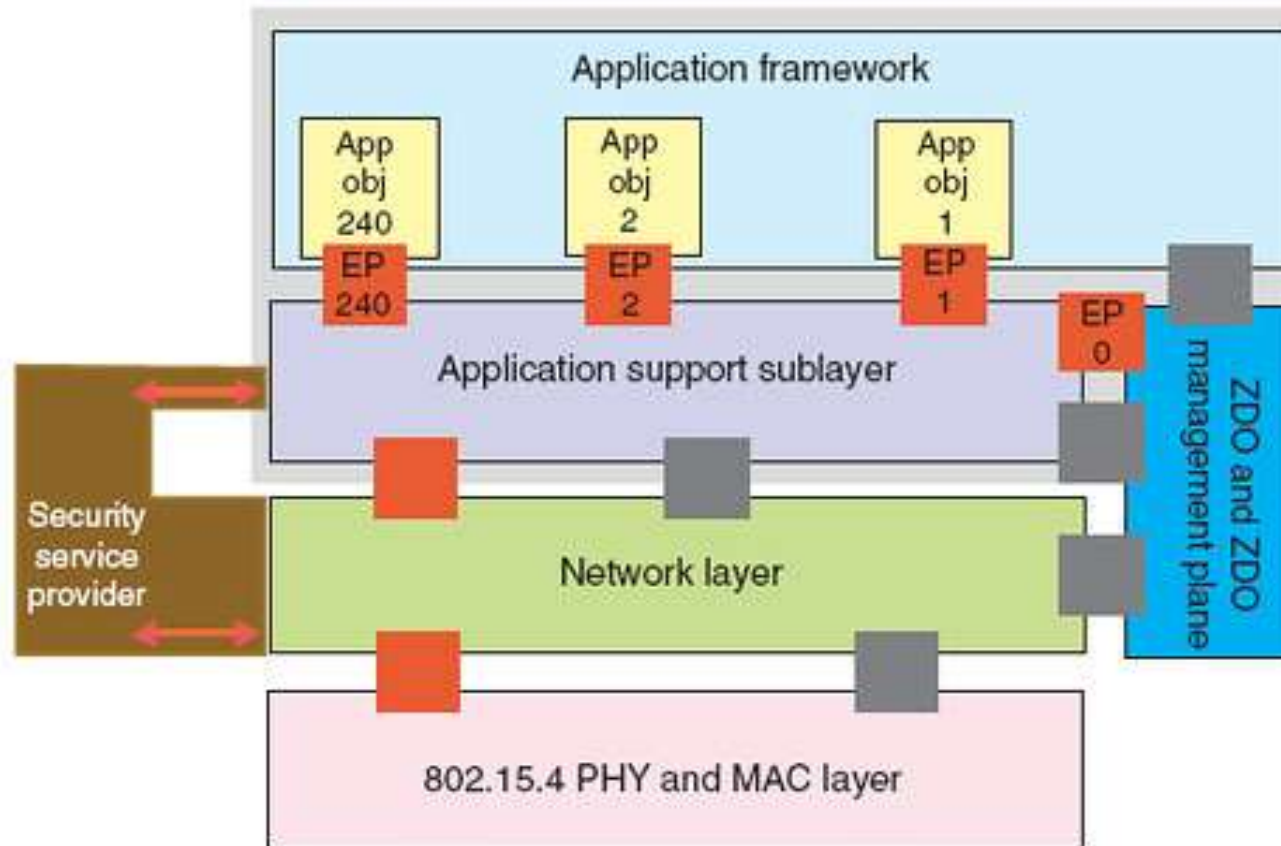
- ❑ GTS- Guaranteed Time Slot

ZIGBEE

ZigBee characteristics

- ☐ Low power consumption – extended battery life
- ☐ Low data rate – 200-250 kbps
 - ☐ Wi-Fi - 11 Mbps, Bluetooth – 1 Mbps
- ☐ Short range
 - ☐ 75-100 meter indoor
 - ☐ Up to 300 meters outdoor
- ☐ Network join time- 30 sec
- ☐ Large and small networks – 65000 (theoretical)
- ☐ Low cost
- ☐ Uses AES encryption for security

ZIGBEE



Zigbee protocol stack

ZIGBEE

IEEE 802.15.4 PHY packet structure

Preamble (32)	Start of packet delimiter (8)	PHY header (8)	PHY Service Data Unit (PSDU) (0-1016 bits)
----------------	----------------------------------	----------------	--

Preamble – synchronization

Start of packet delimiter

PHY header – PSDU length

PSDU – data field

ZIGBEE

IEEE 802.15.4 MAC frame format

- Data frame
- Beacon frame
- Acknowledgement frame
- Command frame

General MAC Frame Format

Octets : 2	1	0/2	0/2/8	0/2	0/2/8	Variable	2
Frame Control	Sequence number	Destination PAN identifier	Destination Address	Source PAN Identifier	Source Address	Frame payload	Frame Check Sequence
MAC header						MAC Payload	MAC footer

ZIGBEE

IEEE 802.15.4 MAC frame format - Beacon Frame Format

Octets : 2	1	4 or 10	2	Variable	variable	Variable	2
Frame control	Beacon sequence number	Source address information	Super frame specification	GTS field	Pending address field	Beacon payload	Frame check sequence
MAC header			MAC payload				MAC Footer

- The beacon frame is transmitted periodically by the PAN coordinator.
- It provides information about the network management through the super frame and GTS fields.
- It also synchronizes the network devices and indicates the proper communication period for them.

ZIGBEE

IEEE 802.15.4 MAC frame format

Command Frame Format

Octets: 2	1	4 to 20	1	Variable	2
Frame control	Data Sequence number	Address information	Command type	Command payload	Frame Check sequence
	MAC header		MAC payload		MAC Footer

- **Useful for communication between the network devices.**
- **The command identifier specifies actions like association, disassociation, routing, and data, GTS or beacon request.**

ZIGBEE

IEEE 802.15.4 MAC frame format

Data Frame Format

Octets : 2	1	4 to 20	Variable	2
Frame Control	Data Sequence number	Address information	Data payload	Frame check sequence
	MAC header		MAC payload	MAC footer

Acknowledgment Frame Format

Octets : 2	1	2
Frame control	Data Sequence Number	Frame Check sequence
	MAC header	MAC footer

ZIGBEE

Parameters	Description
Radio Technology	IEEE 802.15.4
Frequency band/Channels	2.4 GHz (ISM band) 16-channels (2 MHz wide)
Data rate	250 Kbits/sec
Encryption support	AES-128 at Network Layer
Communication range	75-100 meter indoor 300+ meters line of sight
Network size (Theoretical)	Up to 65,000

ZIGBEE

Advantages of ZigBee Protocol

- ❑ Simple setup and maintenance
- ❑ Low power consumption, ideal for long lifetime devices
- ❑ It can be used for lighting, security, appliance and home access
- ❑ Low latency and low data rate
- ❑ It offers network flexibility and scalability

ZIGBEE

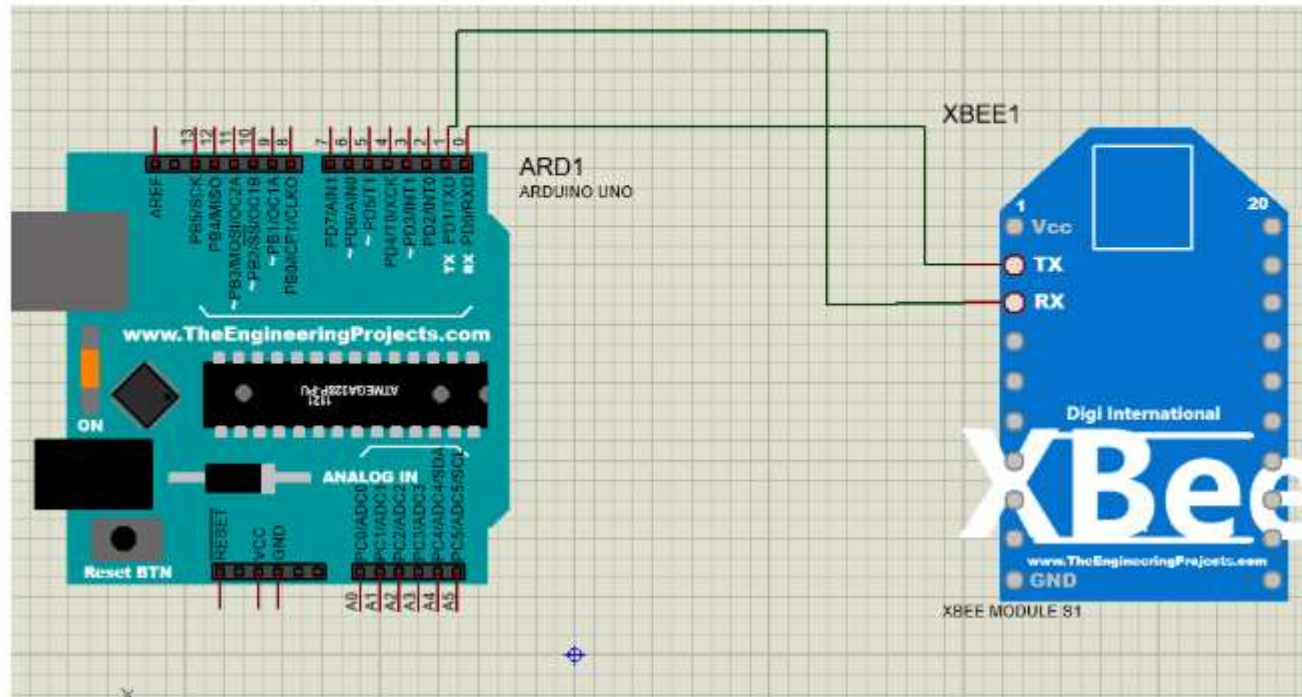
ZigBee connection with Arduino

Gnd Xbee - Gnd Arduino

Vcc Xbee - 3.3V Arduino

Tx - Rx pin D0 Arduino

Rx - Tx pin D1 Arduino



ZIGBEE

Arduino Coding for Zigbee interface

```
#include <SoftwareSerial.h>
SoftwareSerial XBee(2, 3);
void setup()
{
  XBee.begin(9600);
  Serial.begin(9600);
}
void loop()
{
  if (Serial.available())
  {
    XBee.write(Serial.read());
  }
  if (XBee.available())
  {
    Serial.write(XBee.read());
  }
}
```

BLUETOOTH

INTRODUCTION

- ❑ Bluetooth is a transceiver protocol works in 2.4GHz wireless link
- ❑ It's a secure protocol, and it's perfect for short-range, low-power, low-cost, wireless transmissions
- ❑ Zigbee and Wi-Fi both uses same bandwidth
- ❑ Bluetooth is a dynamic standard where devices can automatically find each other, establish connections, and discover what they can do for each other on an ad hoc basis.
- ❑ Bluetooth networks can be referred as piconet and scatternet
- ❑ It use a master/slave model

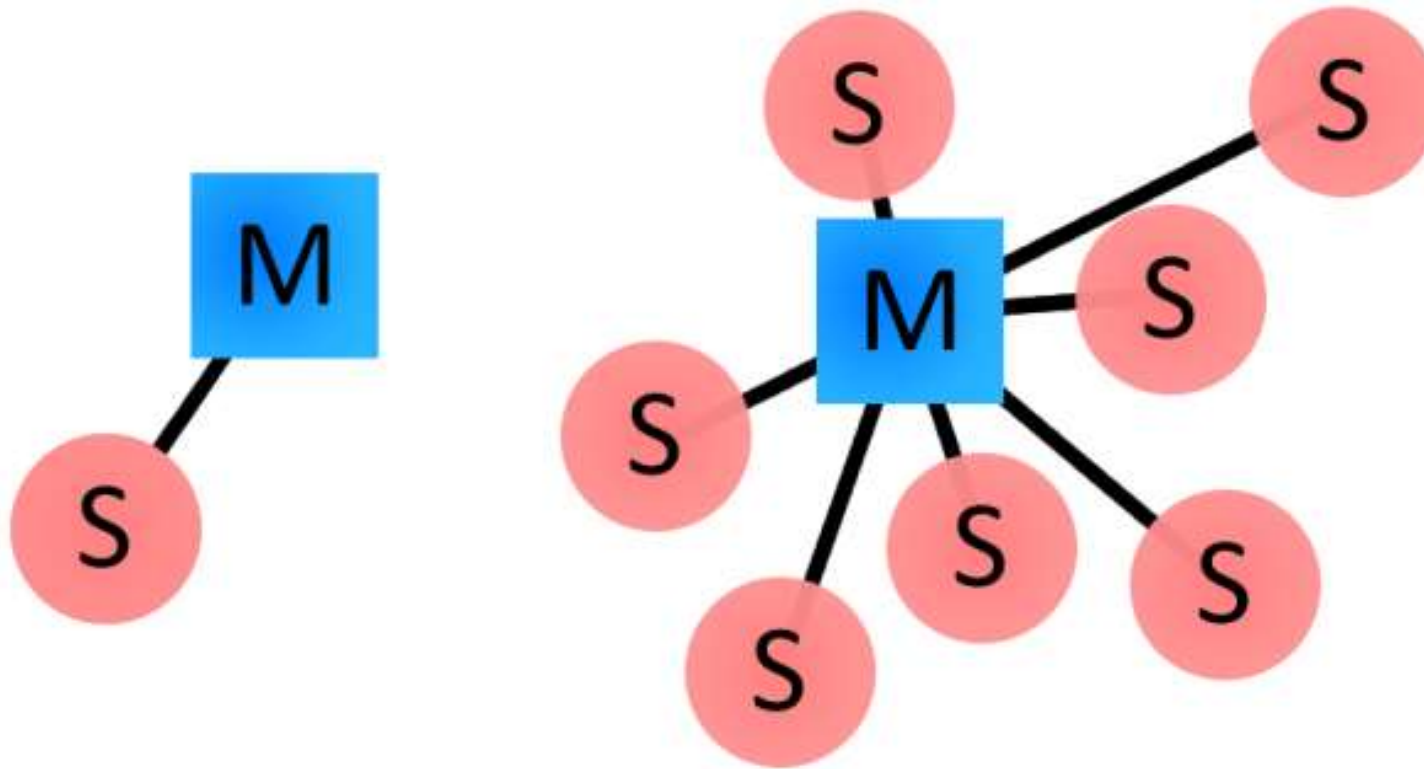
BLUETOOTH

INTRODUCTION

- ❑ A single master device can be connected to up to **seven different slave devices**
- ❑ The master coordinates communication throughout the piconet. It can send data to any of its slaves and request data from them as well
- ❑ **Slaves are only allowed to transmit** to and receive from their master. They can't talk to other slaves in the piconet
- ❑ Every single Bluetooth device has **a unique 48-bit address**, commonly abbreviated BD_ADDR. This will usually be presented in the form of a **12-digit hexadecimal value**

BLUETOOTH

INTRODUCTION



Examples of Bluetooth master/slave piconet topologies.

BLUETOOTH

VERSIONS

Bluetooth 1.0

1998.10 – 2003.11

“Base Rate”

- 1Mbps data rate
- V1.0 - Draft
- V1.0A - published on 1999.7
- V1.0B Enhanced the Interoperability
- V1.1 - IEEE 802.15.1
- V1.2 Enhanced the compatibility

Bluetooth 2.0 + EDR

2004.11 – 2007.7

“Enhanced Data Rate”

- Higher ordered modulation for data payload
- 2Mbps or 3Mbps physical data rate
- V2.0
- V2.1

Bluetooth 3.0 + HS

2009.4

“HS Mode”

- AMP
Alternative MAC/PHY
- Implement high data rate by using 802.11 protocols.
- Facing the Challenge from Wi-Fi
- V3.0

Bluetooth 4.0

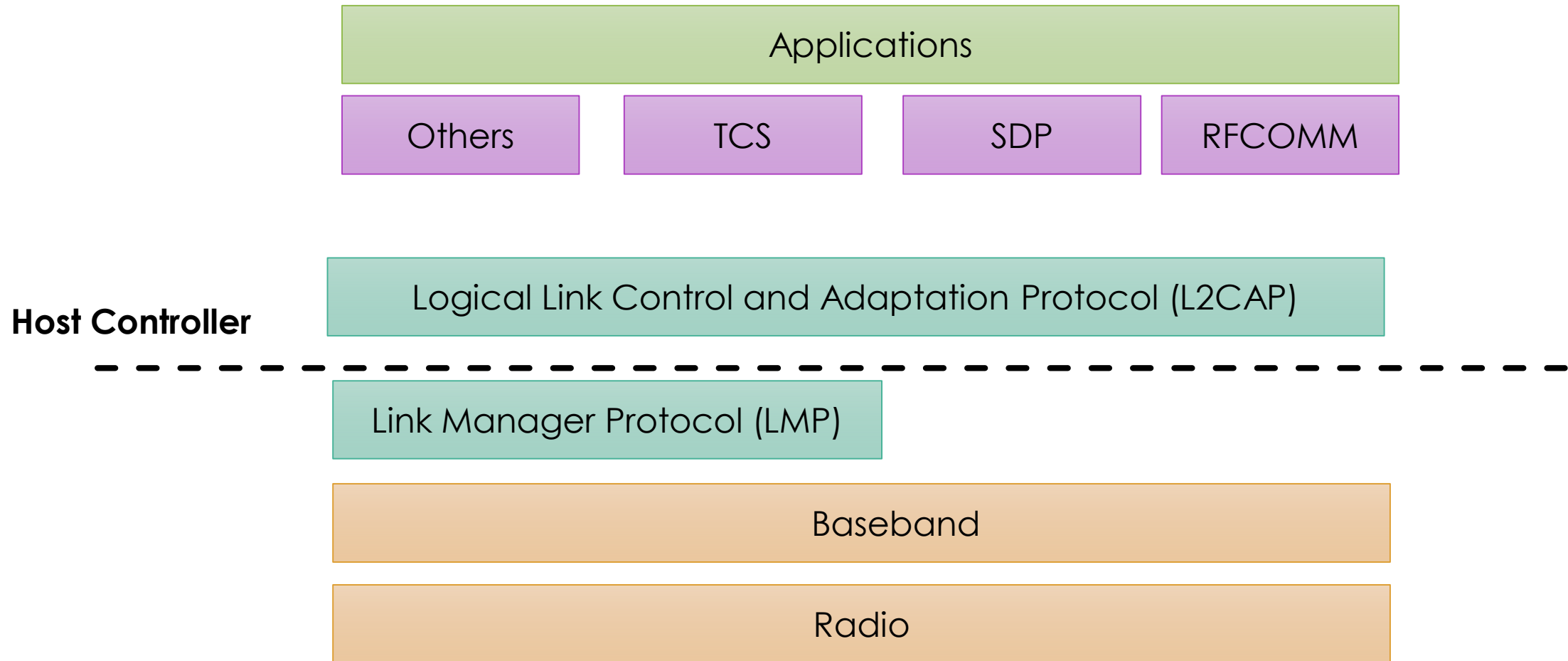
2010.6 – 2014.12

“Low Energy”

- Facing the IoT application
- Changed the protocol greatly, almost a new technology
- V4.0
- V4.1
- V4.2

BLUETOOTH

ARCHITECTURE



BLUETOOTH

ARCHITECTURE

- ❑ **Radio:** This layer defines the requirements for a transceiver to operate in the 2.4 GHz ISM band such as frequency bands, frequency hopping specifications, and modulation techniques.
- ❑ **Baseband:** The Baseband layer describes the specification of the Bluetooth Link Controller (LC), and carries baseband protocol. It defines the addressing scheme, packet frame format, timing, and power control algorithms. Two types of link can be created in baseband are ACL and SCO.

BLUETOOTH

Two types of link between master and slave

- ❑ **Asynchronous Connection Less (ACL)**
 - Packet switched data
 - Slave can have only one ACL link to master
 - Used for correct delivery over fast delivery → high latency
 - Maximum data rate of 721 kbps
- ❑ **Synchronous Connection Oriented (SCO)**
 - Real time data transmission
 - Damaged packet cannot be retransmitted
 - Data rate of 64 kbps
 - fast delivery → low latency

BLUETOOTH

ARCHITECTURE

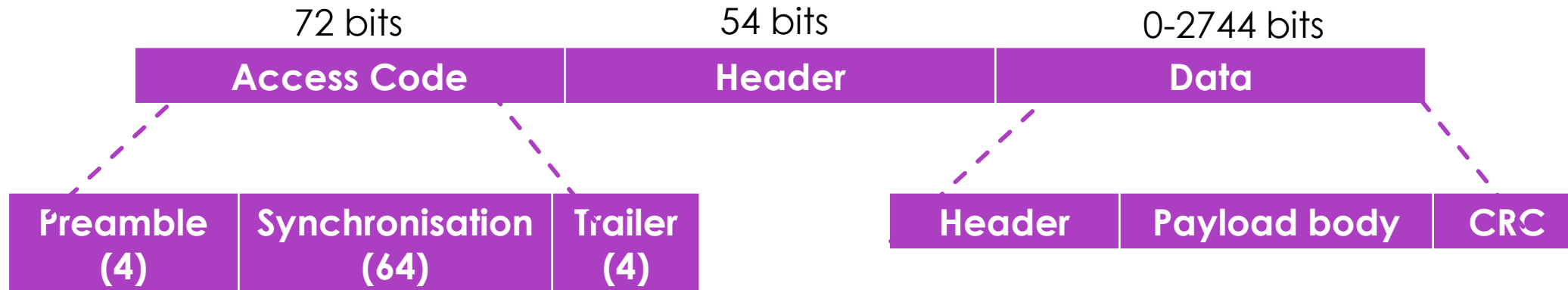
- ❑ **LMP:** Used by the Link managers for link set-up and control. The other main functions of LMP are device authentication, message encryption, and negotiation of packet sizes.
- ❑ **Host Controller Interface (HCI):** HCI provides a command interface to the Baseband LC and Link Manager, to access hardware status and control registers.
- ❑ **L2CAP:** Logical Link Control and Adaptation Protocol (L2CAP) supports higher level protocol multiplexing, packet segmentation and reassembly, and also conveys the QoS.

BLUETOOTH

ARCHITECTURE

- ❑ **RFCOMM:** The RFCOMM protocol provides emulation of serial ports over the L2CAP protocol.
- ❑ **SDP:** The Service Discovery Protocol (SDP) provides a means for applications to discover, which services are provided by or available through a Bluetooth device
- ❑ **TCS:** telephony control protocol for telephony service
- ❑ **Applications:** This includes the application profiles that allow the user to interact with the Bluetooth applications.

BLUETOOTH



Access code (AC)

- Used for packet identification
- Once packet is received, receiver in piconet will compare the incoming signal with AC, if any mismatch is found packet will be ignored
- 72 bit AC is derived from master identity
- It is used for synchronization
- Three types
 - Channel AC – targets piconet ID
 - Device AC – individual devices
 - Inquiry AC – used during pairing

Data - contains data or control information from upper layer

Header – identifies logical channel

Payload body – contains user data

CRC- 16 bit checksum

BLUETOOTH

3 bits	4 bits	1 bits	1 bits	1 bits	8 bits
AM_ADDR	Type	Flow	ARQN	SEQN	HEC

Header

- Address – it can address up to 7 slave devices, if it is 0, then messages will broadcast
- Type – defines type of incoming data
 - ACL – Data medium (DM) or Data High (DH)
 - SCO - Data Voice (DV) and High-Quality Voice (HV)
- 12 types of packet for each SCO and HV
- 4 common control packets
- Flow- 1 bit flow control
- ARQN – 1 bit acknowledgement
- SEQN – 1-bit sequential numbering scheme for packet ordering
- HEC – Header error check
- Header = 3 X 18 bits = 54 bits

BLUETOOTH

BONDING AND PAIRING

- ❑ Pairing requires an **authentication process** to establish connection between devices.
- ❑ Sometimes pairing is a simple **“Just Works”** operation, where the click of a button is all it takes to pair (this is common for devices with no UI, like headsets).
- ❑ Older, legacy (**v2.0 and earlier**), pairing processes involve the entering of a common **PIN code** on each device. The PIN code can range in length and complexity from four numbers (e.g. “0000” or “1234”) to a 16-character alphanumeric string.

BLUETOOTH

CONNECTION PROCESS

- ❑ Creating a Bluetooth connection between two devices is a multi-step process involving three progressive states.
- ❑ **Step-1 Inquiry:** If two Bluetooth devices know absolutely nothing about each other, one must run an inquiry to try to discover the other
- ❑ On receiving the inquiry the device which listens to such a request will send back address, name and other information

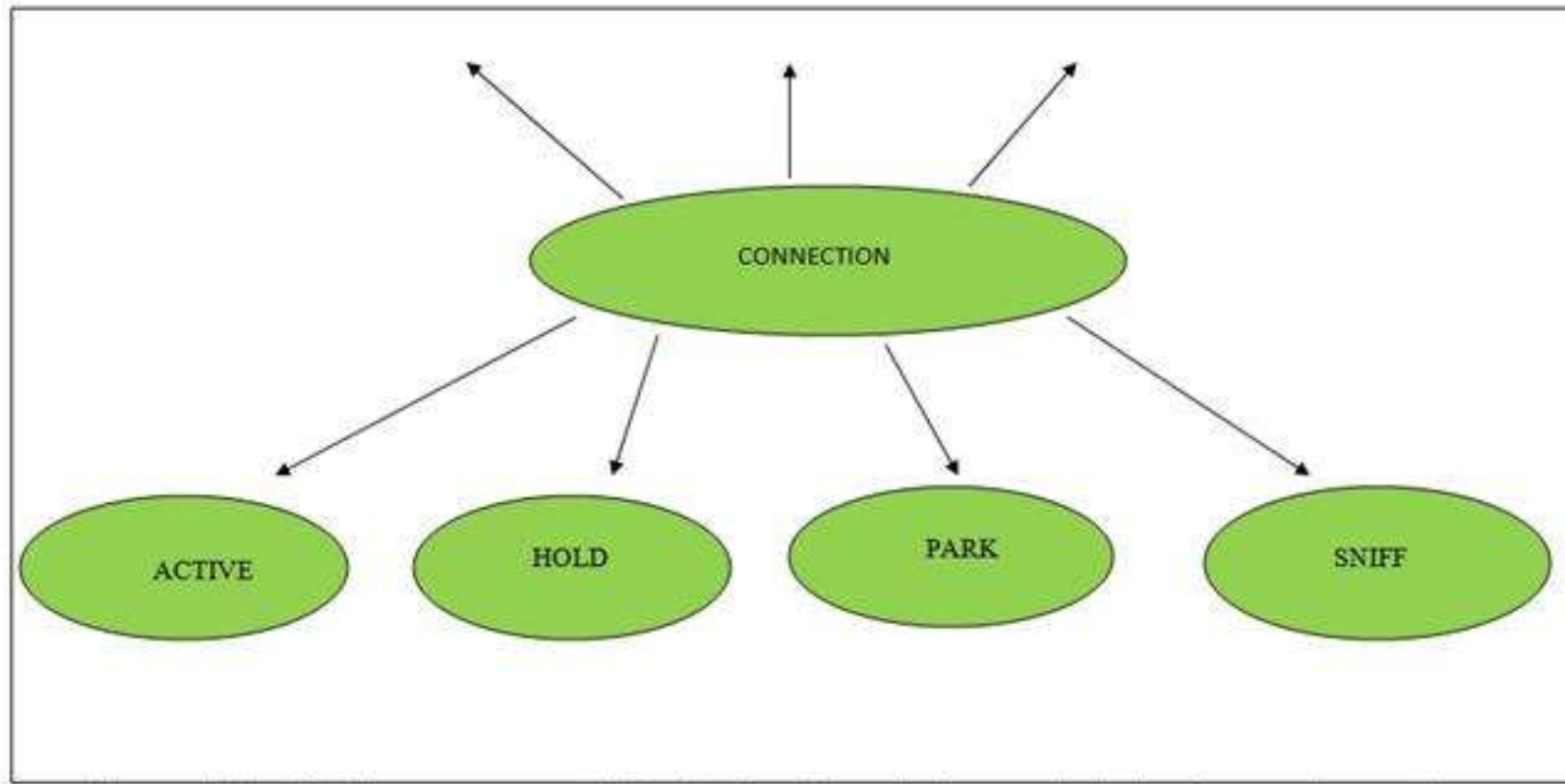
BLUETOOTH

CONNECTION PROCESS

- ❑ **Step-2 Paging (Connecting)** – Paging is the process of forming a connection between two Bluetooth devices.
- ❑ **Step-3 Connection** – After a device has completed the paging process, it enters the connection state.
- ❑ The mode of operation of device is decided in connection phase

BLUETOOTH

CONNECTION MODES



BLUETOOTH

CONNECTION MODES

- ❑ **Active Mode** – regular connected mode, where device send and receive data (regular connection up-to 7 devices)
- ❑ **Sniff Mode** – power saving mode, less active mode (frees slave for predetermined period of time, recurring fixed time slot)
- ❑ **Hold Mode** – temporary mode, device will be sleeping for a particular period of time (frees slave for predetermined period of time, one time slot)
- ❑ **Park Mode** – sleep mode with infinite time, slave will wake only if master tells it to wake up (maximum of 255 devices can be added)

BLUETOOTH

POWER CLASSES

- ❑ The transmit power determines the range of a Bluetooth module and it is defined by its power class.
- ❑ There are three defined classes of power:

Class Number	Max Output Power	Max Output Power	Max Range
Class 1	20 dBm	100 mW	100 m
Class 2	4 dBm	2.5 mW	10 m
Class 3	0 dBm	1 mW	10 cm

BLUETOOTH

HC05 – BLUETOOTH MODULE

- ❑ **HC-05 module is an easy to use Bluetooth SPP (Serial Port Protocol) module, designed for transparent wireless serial connection setup.**
- ❑ **The HC-05 Bluetooth Module can be used in a Master or Slave configuration, making it a great solution for wireless communication.**
- ❑ **This serial port Bluetooth module is fully qualified Bluetooth V2.0+EDR (Enhanced Data Rate) 3Mbps modulation with complete 2.4GHz radio transceiver and baseband.**

BLUETOOTH

HC05 – BLUETOOTH MODULE

- ❑ The Bluetooth module HC-05 is a MASTER/SLAVE module. By default the factory setting is SLAVE.
- ❑ The Role of the module (Master or Slave) can be configured only by AT COMMANDS.
- ❑ Master module can initiate a connection to other devices. The user can use it simply for a serial port replacement to establish connection between MCU and GPS, PC to your embedded project, etc.

BLUETOOTH

HC05 – BLUETOOTH MODULE

☐ Hardware Features

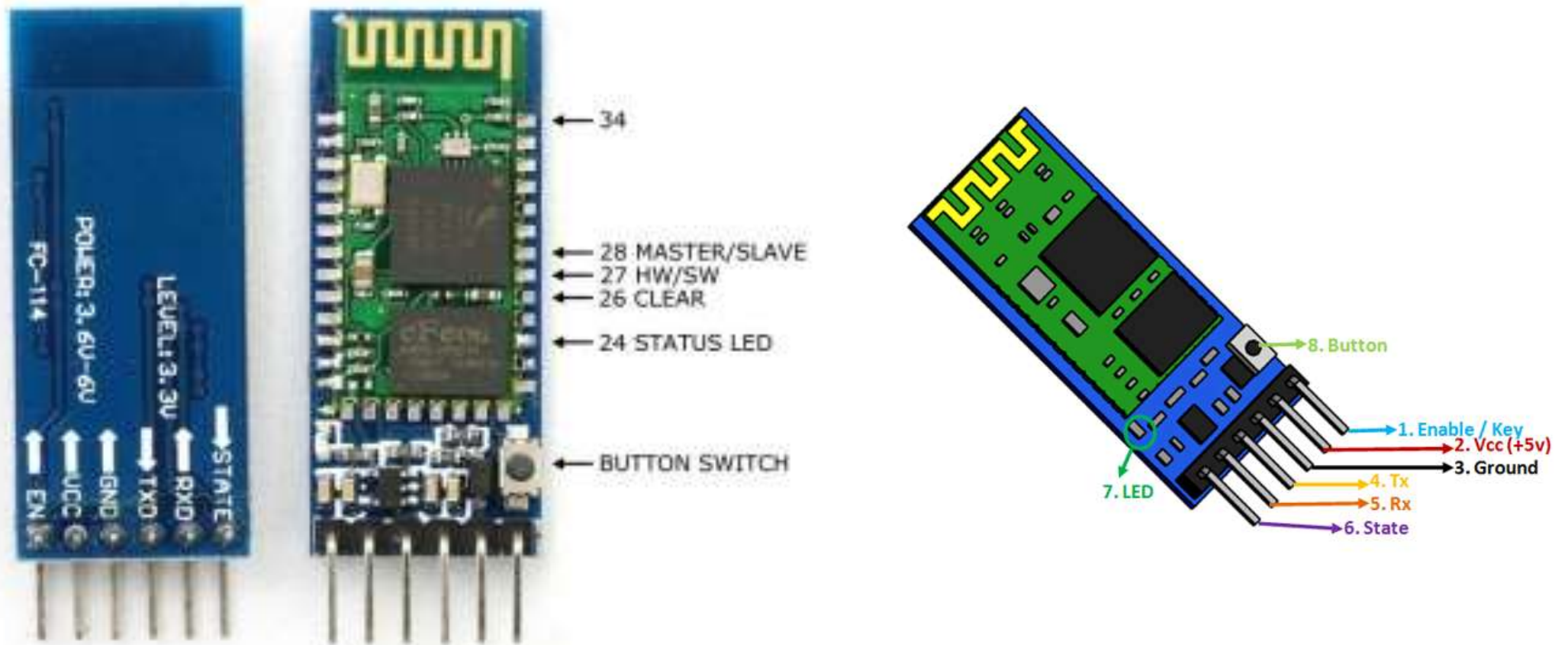
- Up to +4 dBm RF transmit power.
- 3.3 to 5 V I/O.
- PIO(Programmable Input/Output) control.
- UART interface with programmable baud rate.
- With integrated antenna.

☐ Software Features

- Slave default Baud rate: 9600, Data bits:8, Stop bit:1,Parity:No parity.
- Auto-connect to the last device on power as default.
- Permit pairing device to connect as default.
- Auto-pairing PINCODE:"1234" as default

BLUETOOTH

HC05 – BLUETOOTH MODULE



BLUETOOTH

HC05 – BLUETOOTH MODULE – Arduino

```
#define ledPin 7
```

```
int state = 0;
```

```
void setup() {
```

```
  pinMode(ledPin, OUTPUT);
```

```
  digitalWrite(ledPin, LOW);
```

```
  Serial.begin(38400);
```

```
}
```

```
void loop() {
```

```
  if(Serial.available() > 0){ // Checks whether data is coming from the serial port
```

```
    state = Serial.read(); // Reads the data from the serial port
```

```
}
```

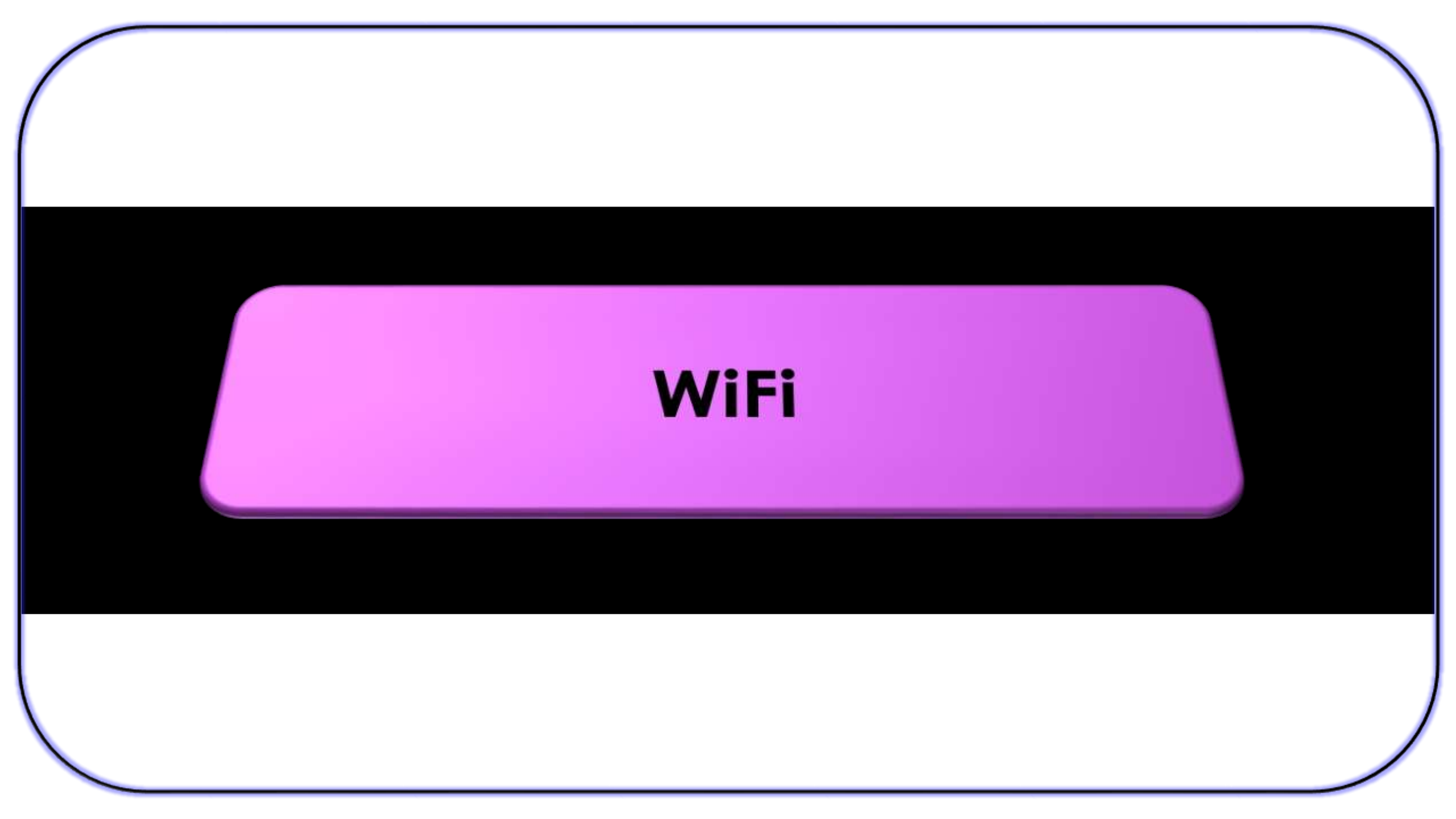
```
  if (state == '0') {
```

Write a program to control a LED connected at Pin 7 of Arduino through command sent by Bluetooth supported mobile device.

BLUETOOTH

HC05 – BLUETOOTH MODULE – Arduino

```
digitalWrite(ledPin, LOW); // Turn LED OFF
Serial.println("LED: OFF"); // Send back, to the phone, the String "LED: ON"
state = 0;
}
else if (state == '1') {
digitalWrite(ledPin, HIGH);
Serial.println("LED: ON");;
state = 0;
}
}
```



WiFi

Wi-Fi

Introduction

- ❑ **WiFi is an alternative network to wired network which is commonly used for connecting devices in wireless mode.**
- ❑ **WiFi stands for Wireless Fidelity. It is a generic term that refers to IEEE802.11 standard for Wireless Local Networks or WLANs.**
- ❑ **WiFi connects computers to each other, to the internet and to the wired network.**

Wi-Fi

WIFI Technology

Standards	Year of Release	Description
Wi-Fi-1 (802.11b)	1999	This version has link speed from 2Mb/s to 11 Mb/s over 2.4 Ghz frequency band
Wi-Fi-2 (802.11a)	1999	After a month of release previous version, 802.11a was released and it provide upto 54 Mb/s link speed over 5 Ghz band
Wi-Fi-3 (802.11g)	2003	In this version the speed was increased up to 54 to 108 Mb/s over 2.4 Ghz
802.11i	2004	This is same as 802.11g but only the security mechanism was increased in this version
802.11e	2004	This is also same as 802.11g, only Voice over Wireless LAN and multimedia streaming are involved
Wi-Fi-4 (802.11n)	2009	This version supports both 2.4 Ghz and 5 Ghz radio frequency and it offers up to 72 to 600 Mb/s speed
Wi-Fi-5 (802.11ac)	2014	It supports speed of 1733 Mb/s in 5 Ghz band

Wi-Fi

Elements of WIFI network

- ❑ **WIFI** uses radio technology to transmit and receive data at high speed.
- ❑ **Access Point (AP)** - The AP is a wireless LAN transceiver or “base station” that can connect one or many wireless devices simultaneously to the Internet.
- ❑ **Wi-Fi cards** - They accept the wireless signal and relay information. They can be internal and external.
- ❑ **Safeguards** - Firewalls and anti-virus software protect networks from uninvited users and keep information secure.

Wi-Fi

Elements of WIFI network

- ❑ **SSID (Service Set Identifier)** : It is a 32-character name which identifies the Wi-Fi network and differentiate one Wi-Fi to another Wi-Fi. All the devices are attempting to connect a particular SSID. Simply, SSID is the name of the wireless network.
- ❑ **WPA-PSK (Wi-Fi Protected Access- Pre-Shared Key)** : It is a program developed by the Wi-Fi Alliance Authority to secure wireless networks with the use of Pre-Shared Key(PSK) authentication. WPA has 3 types, such as WPA, WPA2, WPA3. It is a way of encrypting the Wi-Fi signal to protect from the unwanted users.
- ❑ **Wi-Fi uses Ad-Hoc networks to transmit.** It is a point-to-point network without any interface.

Wi-Fi

WIFI Topologies

- ☐ Peer-to-peer topology (Ad-hoc Mode)
- ☐ AP-based topology (Infrastructure Mode)

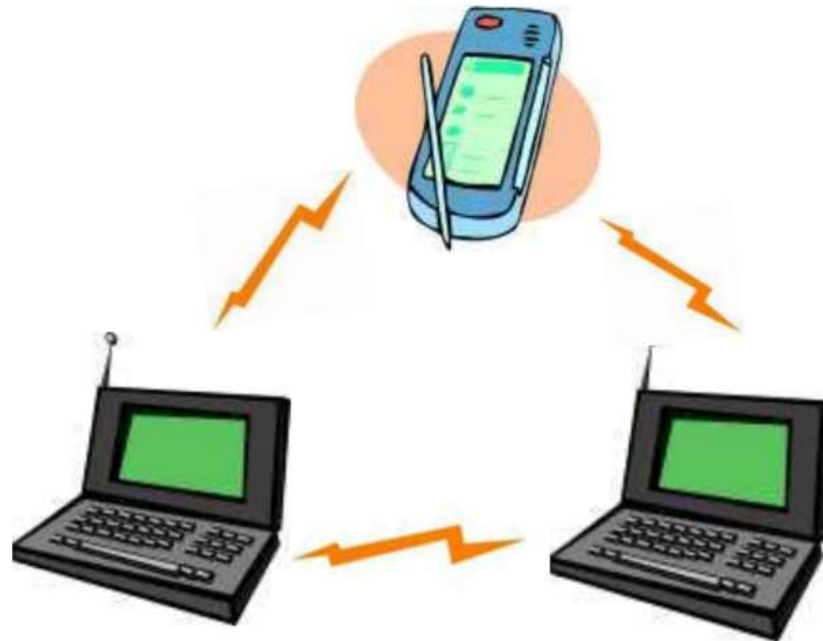
Wi-Fi

Peer-to-peer Topology

- ☐ AP is not required.
- ☐ Client devices within a cell can communicate with each other directly.
- ☐ It is useful for setting up a wireless network quickly and easily.

Wi-Fi

Peer-to-peer Topology



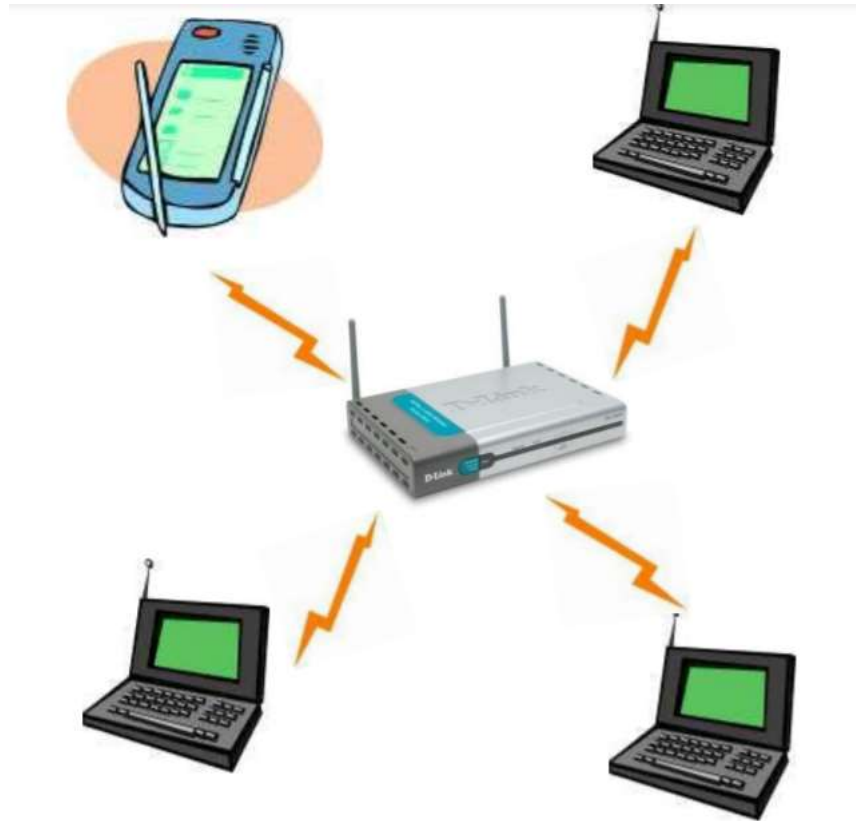
Wi-Fi

Infrastructure network

- ❑ The client communicate through Access Point.
- ❑ Any communication has to go through AP.
- ❑ If a Mobile Station (MS), like a computer, a PDA, or a phone, wants to communicate with another MS, it needs to send the information to AP first, then AP sends it to the destination MS.

Wi-Fi

Infrastructure network



Wi-Fi

Hotspots

- ❑ A Hotspot is a geographical area that has a readily accessible wireless network.
- ❑ Hotspots are equipped with Broad band Internet connection and one or more Access points that allow users to access the internet wirelessly.
- ❑ Hotspots can be setup in any public location that can support an Internet connection. All the locations discussed previously are examples of Hotspots.

Wi-Fi

How a Wi-Fi Network Works

- ❑ A Wi-Fi hotspot is created by installing an access point to an internet connection.
- ❑ An access point acts as a base station.
- ❑ When Wi-Fi enabled device encounters a hotspot, the device can then connect to that network wirelessly.

Wi-Fi

How a Wi-Fi Network Works

- ❑ A single access point can support up to 30 users and can function within a range of 100 – 150 feet indoors and up to 300 feet outdoors.
- ❑ Many access points can be connected to each other via Ethernet cables to create a single large network.

Wi-Fi

Advantages

- ☐ Mobility
- ☐ Ease of Installation
- ☐ Flexibility
- ☐ Cost
- ☐ Reliability
- ☐ Security
- ☐ Use unlicensed part of the radio spectrum
- ☐ Roaming
- ☐ Speed

Wi-Fi

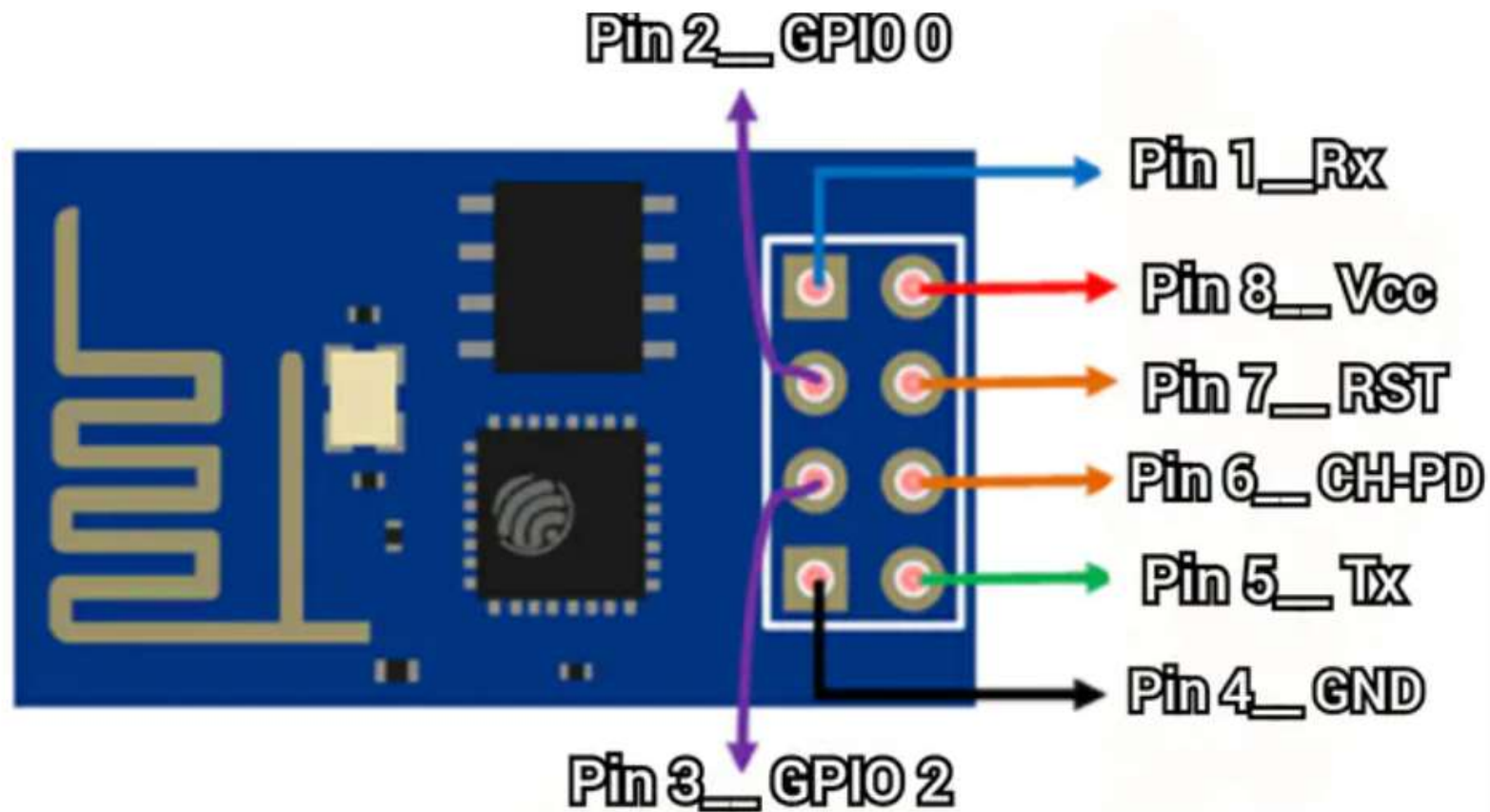
Limitations

- ☐ Interference
- ☐ Degradation in performance
- ☐ High power consumption
- ☐ Limited range

Wi-Fi

Interface with Arduino

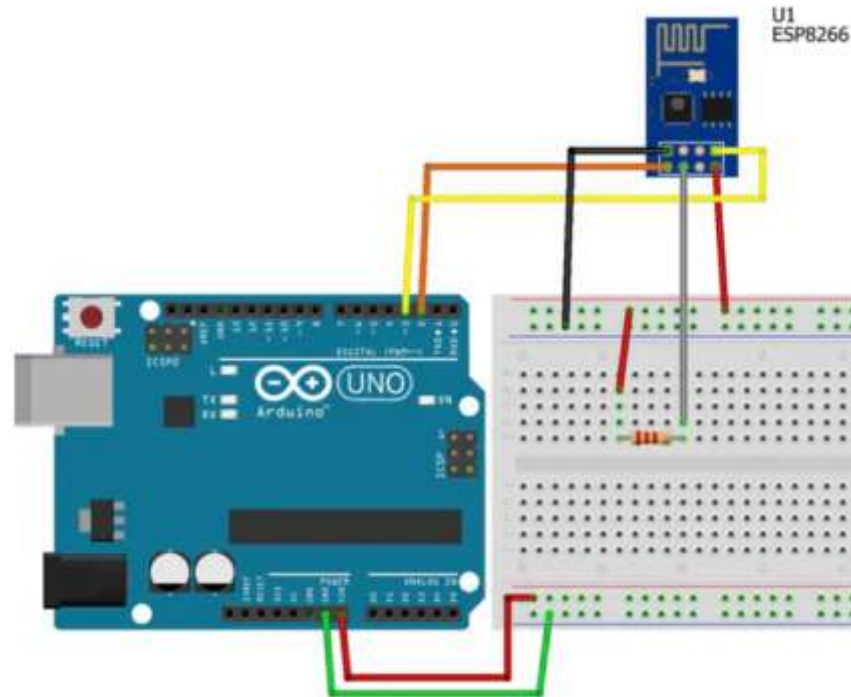
❑ ESP8266



Wi-Fi

Interface with Arduino

❑ ESP8266



Wi-Fi

Interface with Arduino

```
#include <SoftwareSerial.h>
SoftwareSerial ESPserial(2, 3); // RX | TX
void setup()
{
  Serial.begin(115200); // communication with the host computer
  //while (!Serial) { ; }
  // Start the software serial for communication with the ESP8266
  ESPserial.begin(115200);
  Serial.println("");
  Serial.println("Remember to set Both NL & CR in the serial monitor.");
  Serial.println("Ready");
  Serial.println("");
}
```

Wi-Fi

Interface with Arduino

```
void loop()
{
  // listen for communication from the ESP8266 and then write it to the serial monitor
  if ( ESPserial.available() ) { Serial.write( ESPserial.read() ); }
  // listen for user input and send it to the ESP8266
  if ( Serial.available() ) { ESPserial.write( Serial.read() ); }
}
```

INTRODUCTION.

- ❑ **RS-232** (**Recommended Standard 232**).
- ❑ It is basically a interface standards.
- ❑ It is commonly used in computer **serial ports**.
- ❑ The standard defines the electrical characteristics and timing of signals.
- ❑ The current version of the standard is **TIA-232-F**, issued in 1997.

History.

- RS-232 was first introduced in 1962.
- For many years, an RS-232-compatible port was a standard feature for serial communication.
- The standard continued to be revised and updated by the Electronic Industries Alliance and since 1988 by the Telecommunications Industry Association (TIA).
- The C revision of the standard was issued in August 1969.



RS-232 Port.

STANDARD.

- ❖ In RS-232, user data is sent as a time-series of bits.
- ❖ Both synchronous and asynchronous transmissions are supported by the standard.
- ❖ RS-232 devices may be classified as Data Terminal Equipment (DTE) or Data Communication Equipment (DCE), this defines that each device which wires will be sending and receiving each signal.
- ❖ DTE refers to terminals and computers that sends and receives data.
- ❖ DCE refers to communication equipment, such as modems, that are responsible for data transferring.

Limitations of the Standard.

- The limited transmission speed, relatively large voltage swing, and large standard connectors motivated development of the universal serial bus(USB) which has displaced RS-232
- Multi-drop connection among more than two devices is not defined.
- Also multi-drop have limitations in speed and compatibility.

DIFFERENT TYPES OF RS-232 CABLES.



RS-232 used in.....

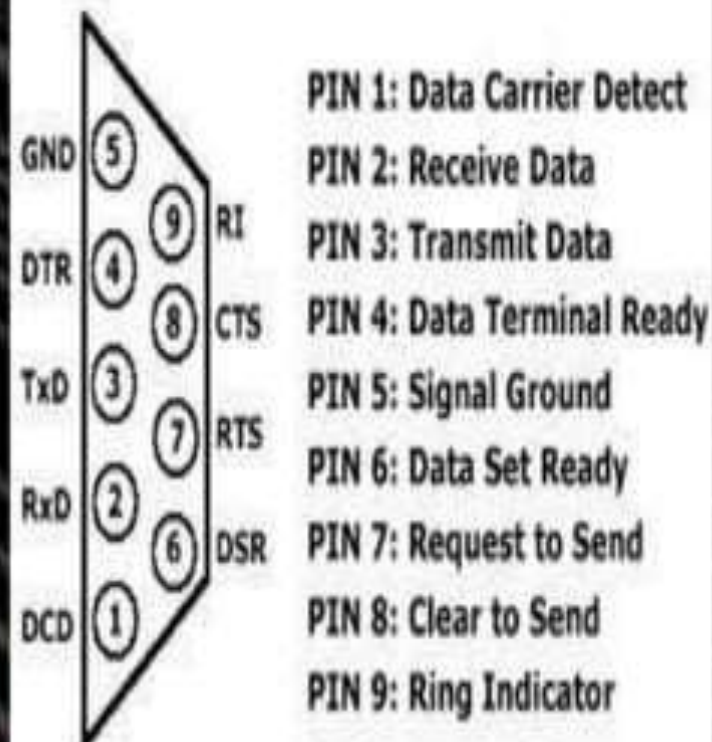


RS-232 pins.

RS232 Pinout on DB25



RS-232 DB-9 Male Pinout



Examining RS-232 handshaking signals.

- ✓ To ensure fast and reliable data transmission between two devices, the data transfer must be coordinated.
- ✓ Many of the pins of RS-232 connector are used for handshaking signals.

PINS OF RS232

1. DTR(Data Terminal Ready)
 2. DSR(Data Set Ready)
 3. RTS(Request to Send)
 4. CTS(Clear to Send)
 5. DCD(Data Carrier Detect)
 6. RI(Ring Indicator)
- 

REFERENCES.

- 1) Muhammad Ali Mazidi & Janile Gillispie Mazidi.
- 2) www.wikipedia.com
- 3) www.yahooanswers.com